

软件工程

软件测试

刘 爽

中国人民大学

本课程部分教学内容基于：《软件工程-理论与实践》，毛新军、董威编著，高等教育出版社，2024.1，ISBN 978-7-04-061010-9 配套课件由出版社/作者提供，仅用于教学

内容

1. 软件测试概述

- ✓ 软件测试的思想和原理

2. 软件测试的过程和策略

- ✓ 软件测试的活动及实施的方法

3. 软件测试技术

- ✓ 白盒和黑盒测试技术

4. 软件测试计划及输出

- ✓ 测试计划制定及测试结果



1.1 示例：软件中存在缺陷，导致软件失效

高校课程实践社区

当前位置：主页 > 课程实践平台 > 软件工程



ID:342

[配置](#) [关闭](#)

软件工程

教师 (2) | 学生 (21) | 资源 (2)

主讲教师：毛新军
学时总数：54 学时
课程学期：2015 秋季学期
开设单位：国防科学技术大学

动态 (15)

课程作业 (0)	+发布作业
课程通知 (0)	+发布通知
资源库 (2)	+上传文件
讨论区 (7)	+发布新帖
留言 (0)	
问卷调查 (1)	+新建问卷

课内搜索 全站搜索

上传： [课件](#) | [软件](#) | [媒体](#) | [代码](#) | [其他](#)

共有 2 个资源

按 时间 / 下载次数 / 引用次数 排序

课件 x2

2 软件与软件工程.pdf

选入我的其他课程

公开

预览

文件大小：4.475 MB

1天之前 | 下载2 | 引用0 | 删除

课件 x + 添加标签

1 课程介绍与要求.pdf

选入我的其他课程

公开

预览

文件大小：6.755 MB

6 天之前 | 下载10 | 引用0 | 删除

课件 x + 添加标签

软件缺陷示例：

- ✓ 上传资源后资源数量没有变化
- ✓ 查找以前上传的资源找不到

示例：软件中存在缺陷，导致软件失效

 学习空间

搜索空间

意见反馈

+

31



 软件工程课程综合实践（学生）

资源区 286

讨论区 25

贡献区 118

设置



软件工程课程综合实践（学生）

围绕软件工程课程综合实践，针对综合实践中的需求分析、软件设计、系统建模、编码测试、软件维护等方面的内容，讨论问题，分享成果，交流经验，共享资源

37

28

资源区

新建资源

新建目录

资源名

资源分享

技术博客

更新时间

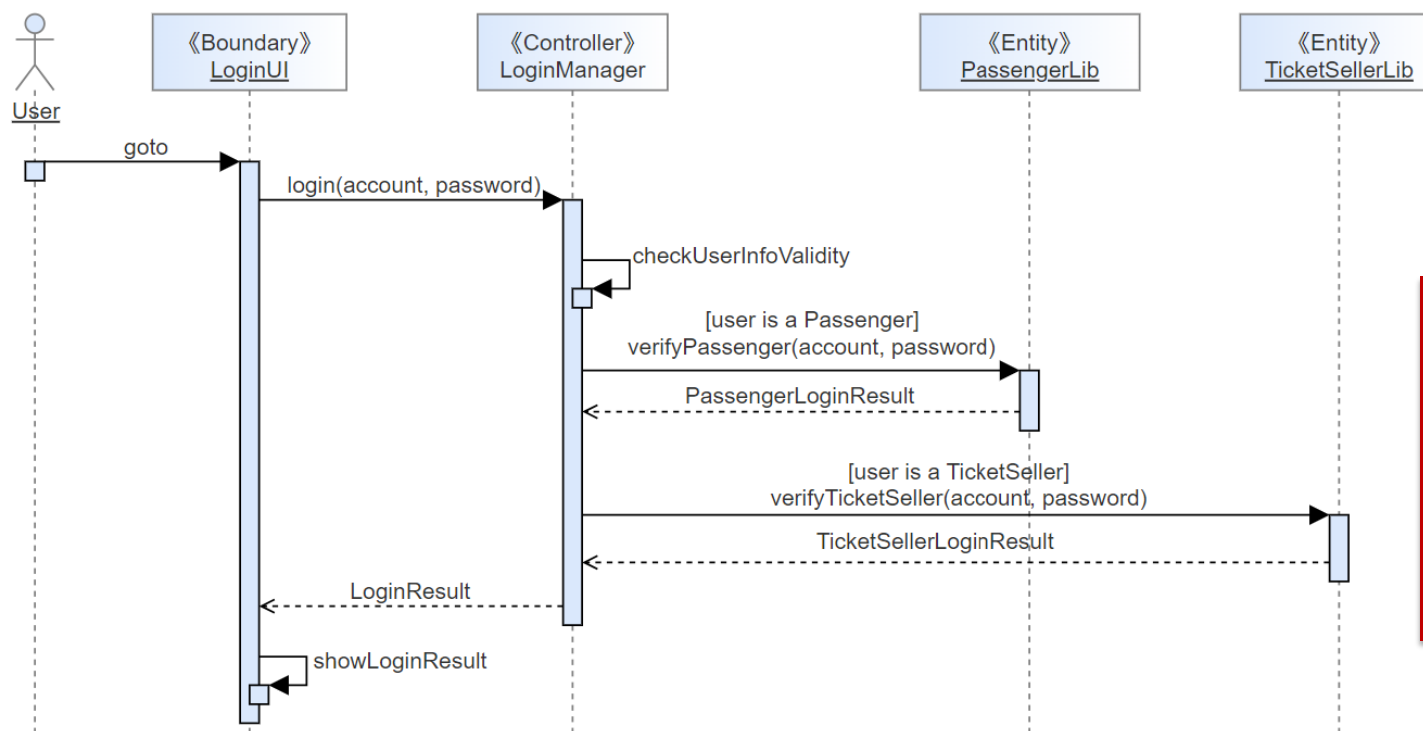
1 天前

7 天前

软件缺陷示例：

- ✓ 注册成功却无法登陆
- ✓ 增加资源但没有显示
- ✓

示例：软件需求中的潜在问题

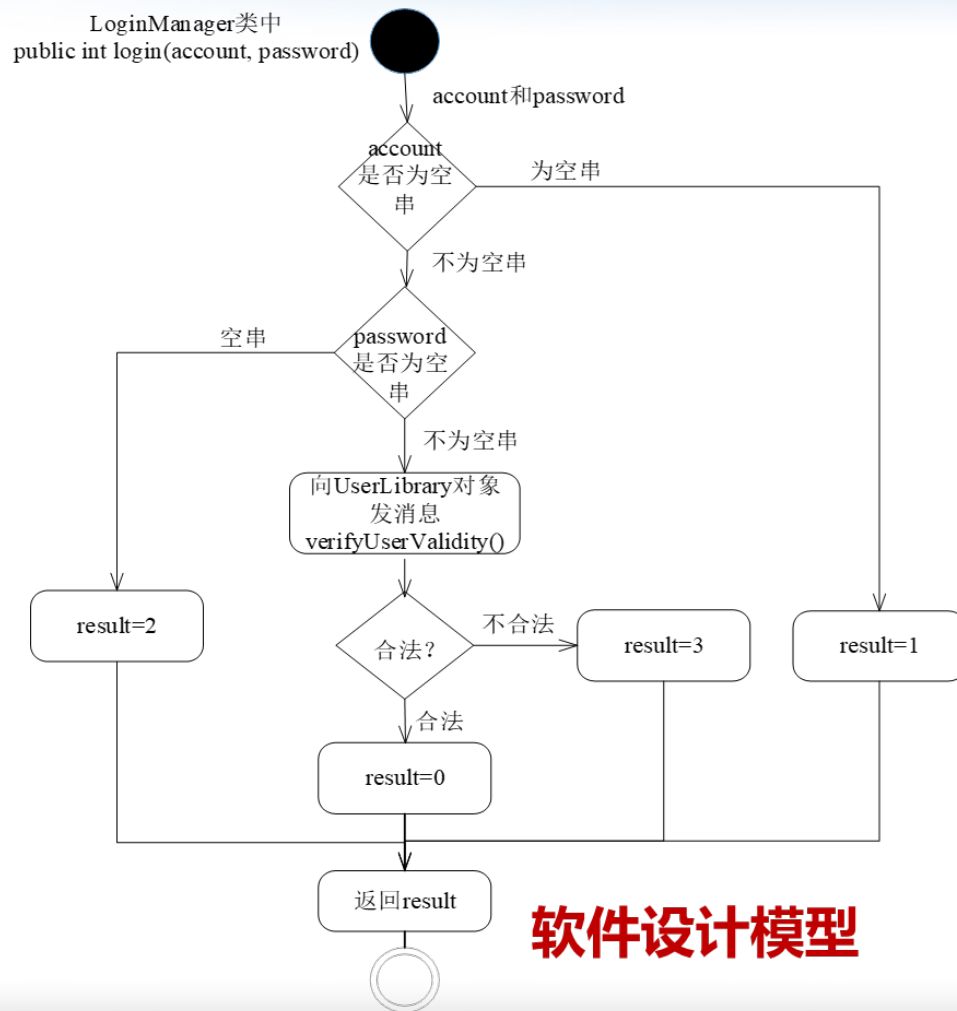


软件需求缺陷：

- 没有正确描述用户需求？
- 没有准确地描述需求？
-

软件需求模型

示例：软件设计中的潜在问题



软件设计存在缺陷

- 没有正确实现软件需求？
- 软件设计存在问题？

示例：程序代码中的潜在问题

```
Notes x Contact.java x
28 public class Contact {
29     private static HashMap<String, String> sContactCache;
30     private static final String TAG = "Contact";
31
32     private static final String CALLER_ID_SELECTION = "PHONE_NUMBERS_EQUAL(" +
Phone.NUMBER
33     + ",?) AND " + Data.MIMETYPE + "='" + Phone.CONTENT_ITEM_TYPE + "'"
34     + " AND " + Data.RAW_CONTACT_ID + " IN "
35         + "(SELECT raw_contact_id "
36         + " FROM phone_lookup"
37         + " WHERE min_match = '+)";
38
39     public static String getContact(Context context, String phoneNumber) {
40         if(sContactCache == null) {
41             sContactCache = new HashMap<String, String>();
42         }
43
44         if(sContactCache.containsKey(phoneNumber)) {
45             return sContactCache.get(phoneNumber);
46         }
47     }
48 }
```

程序代码中的缺陷

- 代码没有正确实现功能？
- 代码编写不正确？



需求和设计缺陷问题会带到代码中，编码时也会引入问题，导致代码存在缺陷

1.2 软件缺陷的危害

- 无法满足要求
- 不能正常工作
- 引发安全事故
- 影响人员安全
- 产生经济损失
-



**波音737
MAX事件**



**原软件用于教练飞机
某参数屏显范围
 ± 100 ，重用于新战
斗机，该参数屏显范
围应该为 ± 300 ！**

尽可能减少软件缺陷非常重要

1.2 软件缺陷不可避免

□人总是会犯错误的

- ✓软件工程师、用户等
- ✓软件系统太复杂

□程序缺陷来自多个源头

- ✓需求、设计、编码活动
- ✓模型、文档、程序制品

□缺陷成常态化

- ✓对于复杂软件系统而言缺陷不可避免
- ✓很难做到无缺陷的软件

你所使用的软件中是否发现有缺陷？

思考和讨论

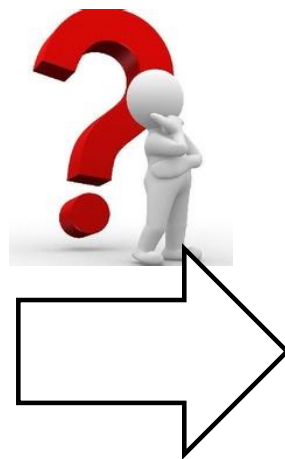
□如何应对软件缺陷？

- ✓如何**避免和减少**缺陷？
- ✓如何**发现和修复**缺陷？



如何应对软件缺陷?

能否有效和快速地发现软件缺陷



方法: 软件测试
(Software Testing)

软件测试方法可以帮助发现软件中潜藏的缺陷

1.3 何为软件测试？

□ **软件测试是运行软件或模拟软件的执行，发现软件缺陷的过程**

□ 注意点

- ✓ 软件测试通过**运行程序代码**的方式来发现程序代码中潜藏的缺陷，这和**代码走查、静态分析**形成鲜明对比。
- ✓ 软件测试的目的是为了**发现软件中的缺陷**。它只负责发现缺陷，不负责修复和纠正缺陷

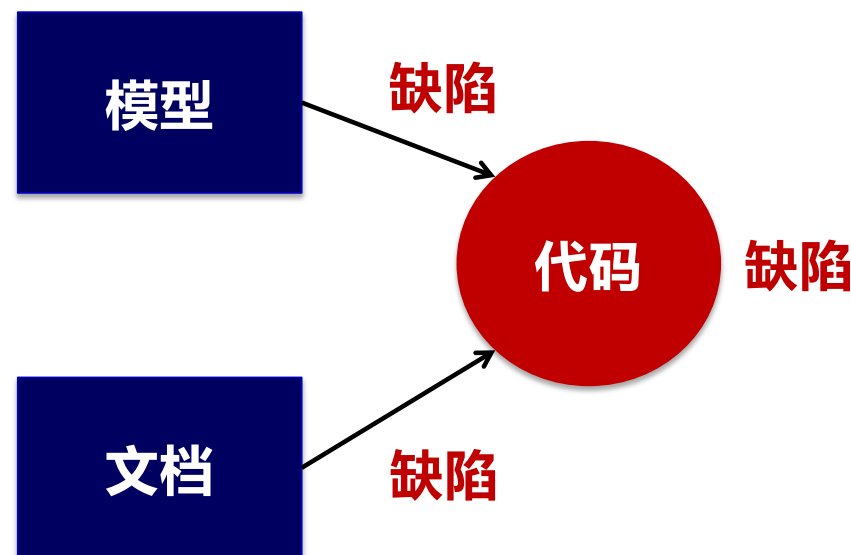
在程序代码中找出软件缺陷

□程序是运行软件的载体

- ✓通过执行代码运行软件
- ✓通过软件运行发现缺陷

□程序是软件缺陷的载体

- ✓缺陷分布在模型、文档和代码中
- ✓最终会反映在程序代码上



需求和设计中存在的缺陷最终会反映在程序代码之中

思考和讨论

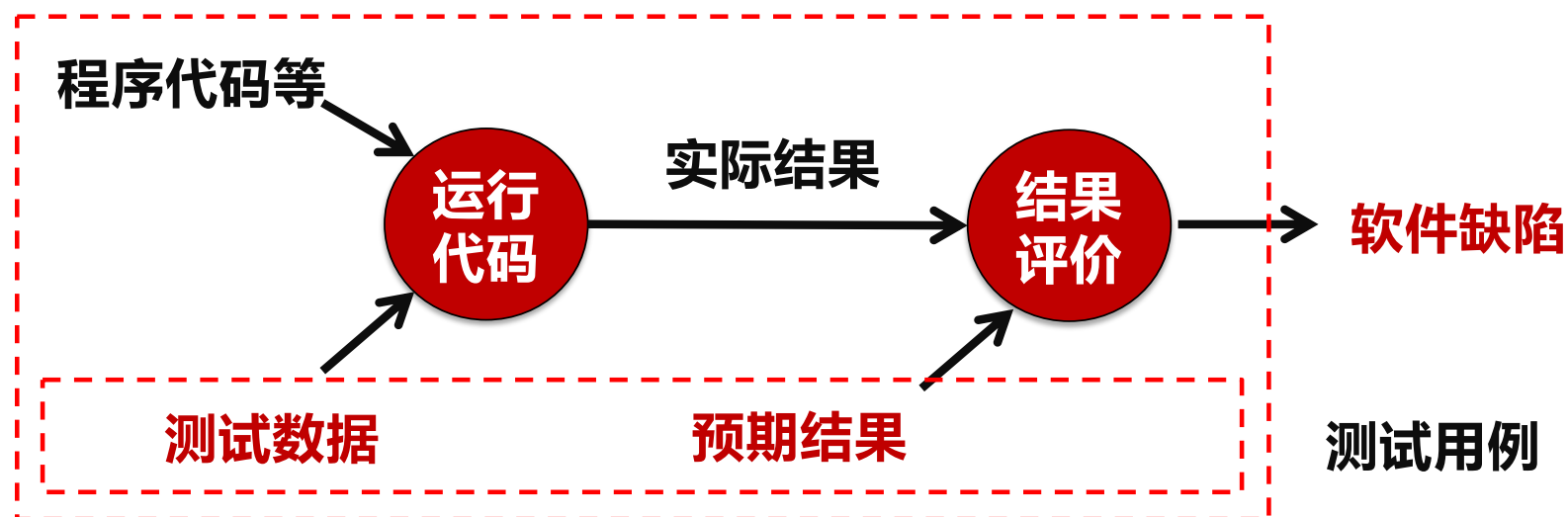
□如何从程序代码中找出软件缺陷？



1.4 软件测试的原理

□**前提认识**：程序本质上是对数据的处理

□**方法思路**：设计数据(测试用例) → 运行测试用例(程序来处理数据) → 判断运行结果(是否符合预期结果)



为软件测试而设计的数据称为测试用例(Test Case)

示例：软件测试的原理

□加法的功能

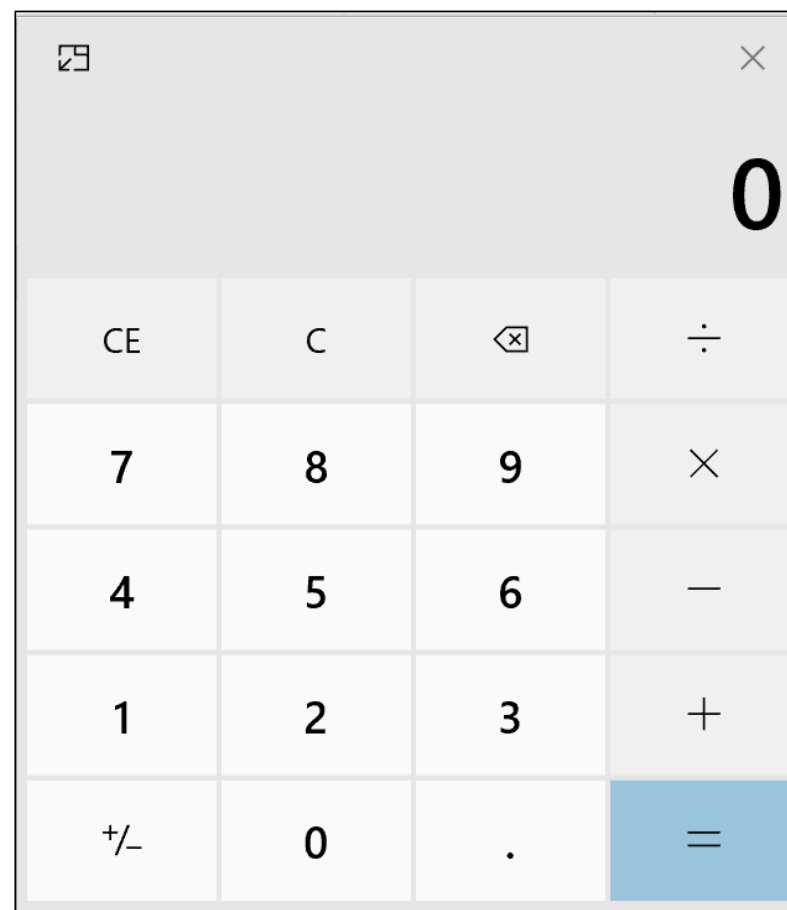
□ $A = B + C$

□测试用例

✓测试数据 1, 2, 预期结果 3

□运行程序及测试用例

✓判断运行的实际结果是否与预期结果相一致



测试用例

□测试用例是一个四元偶

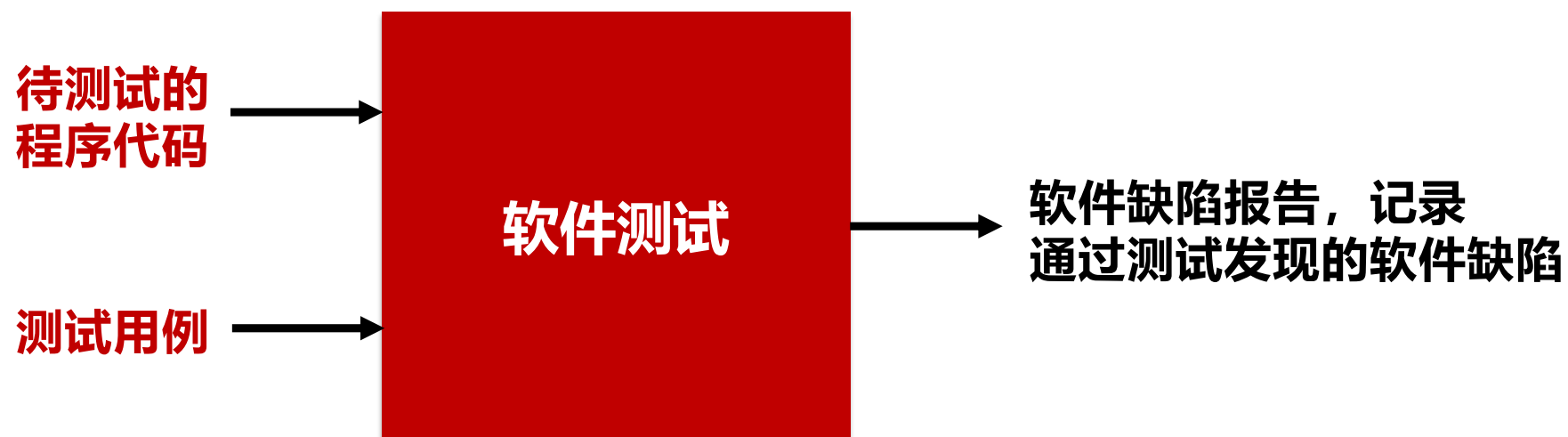
- ✓ **输入数据**：交由待测试程序代码进行处理的数据
- ✓ **前置条件**：程序处理输入数据的运行上下文，即要满足前置条件
- ✓ **测试步骤**：程序代码对输入数据的处理可能涉及到一系列的步骤，其中的某些步骤需要用户的进一步输入
- ✓ **预期输出**：程序代码的预期输出结果

示例：测试用例的设计

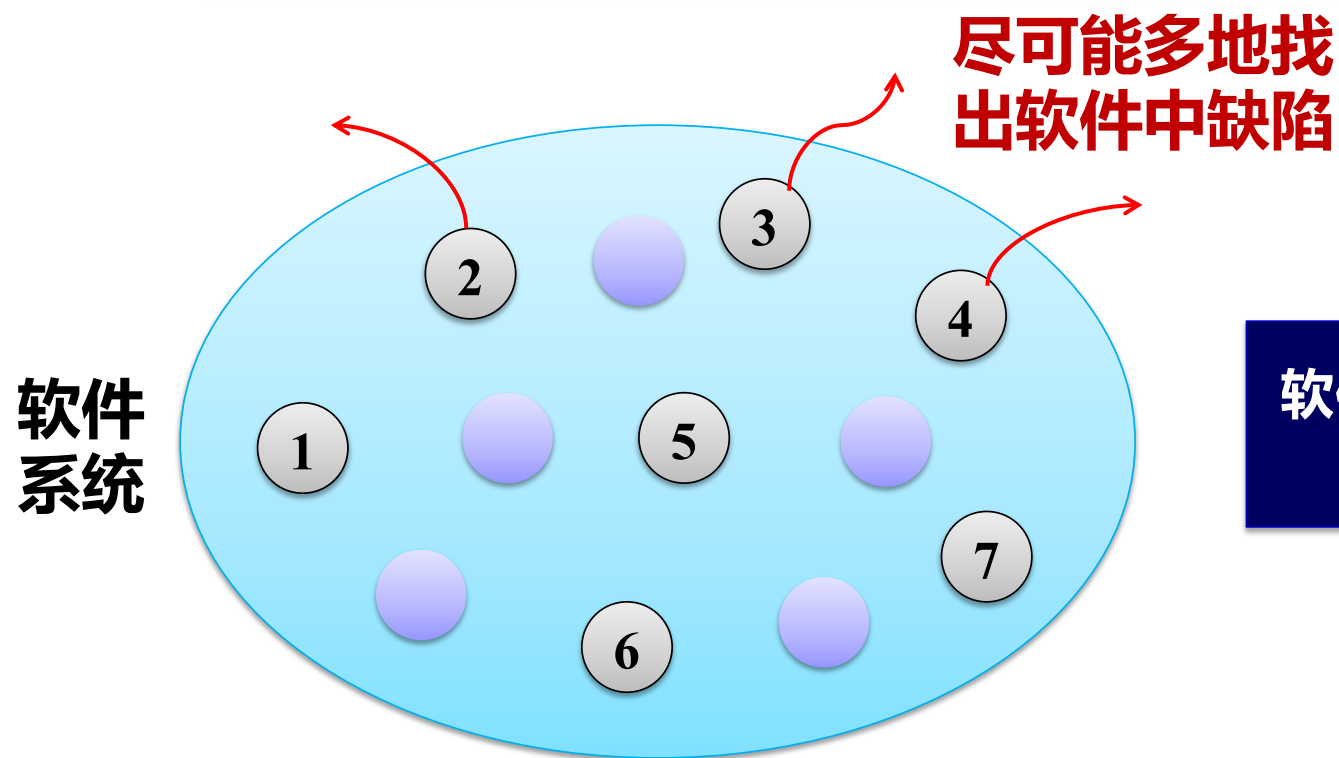
□ “用户登录” 模块单元的测试用例设计

- ✓ **输入数据**：用户账号= “admin” ， 用户密码= “1234”
- ✓ **前置条件**：用户账号 “admin” 是一个尚未注册的非法账号，也即 “T_User” 表中没有名为 “admin” 的用户账号。
- ✓ **测试步骤**：首先清除 “T_User” 表中名为 “admin” 的用户账号；其次用户输入 “admin” 账号和 “1234” 密码；第三，用户点击界面的确认按钮；最后，系统提示 “用户无法登录系统” 的信息
- ✓ **预期输出**：系统将提示 “用户无法登录系统” 的提示信息

1.5 软件测试的任务



软件测试任务



软件缺陷可能隐藏的比较深，
难以发现

能否通过软件测试的方法把软件中的所有缺陷都找出来？



软件测试的目的

□目的

- ✓发现软件中的缺陷
- ✓最大限度、尽可能多的找到缺陷

□功效

- ✓发现的缺陷越多 → 软件中遗留的缺陷越少
- 交付的软件质量越高 → 后期维护工作量就越少

思考和讨论

- 软件测试**没有发现缺陷**是否意味着软件就没有缺陷？为什么？
- 软件测试方法能够用于**证明软件无缺陷**吗？



1.6 软件测试的步骤

① 明确待测试对象

✓ 什么粒度的程序代码

② 设计测试用例

✓ {<Data, Result>}

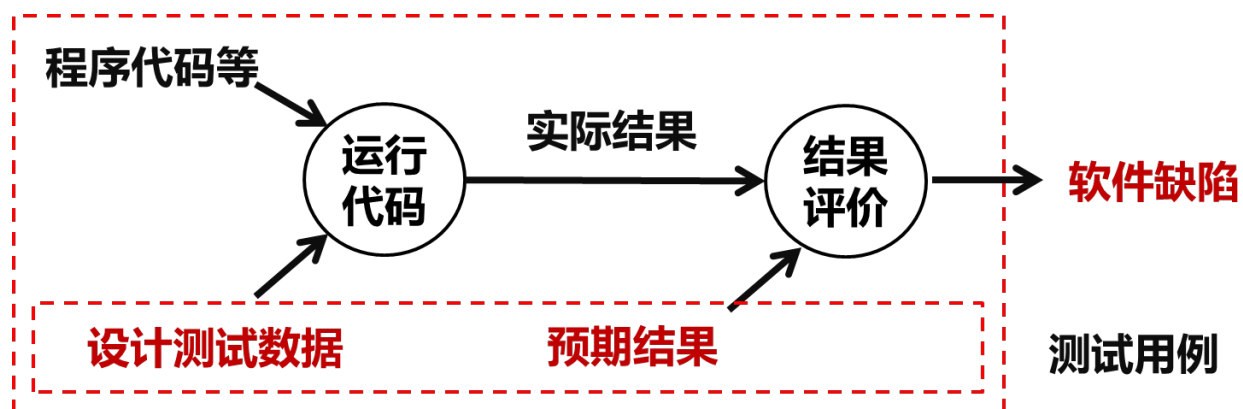
✓ 可能有许多

③ 运行代码和测试用例

✓ 输入和处理测试用例

④ 分析运行结果

✓ 对比运行结果和预期结果，发现问题和缺陷



示例：软件测试的步骤

□明确测试对象

- ✓完成用户注册功能的程序模块（类）

□设计测试用例

- ✓输入：UserID = xjmao, Psw=se
- ✓预期：在数据库中有该用户密码的数据条目

□运行测试用例

- ✓运行程序，输入数据

□分析运行结果

- ✓用户数据库表中是否有该新用户的密码信息

1.7 软件测试面临的主要挑战

□设计测试用例

- ✓C1: 如何设计有效的测试用例, 提高软件测试的质量
- ✓C2: 如何确保测试用例的合理性, 尽可能地发现缺陷?

□发现程序缺陷

- ✓C3: 如何运行程序和用例来发现软件缺陷?
- ✓C4: 如何采用工具来自动地发现缺陷, 提高测试的效率?

软件测试的前提和关键是要设计出有效的测试用例

1.8 软件测试工程师

- **负责软件系统的测试工作，发现软件系统中的缺陷，协助软件开发工程师定位和修复缺陷**
 - ✓ 服务于软件开发工程师，帮助他们理解和解决软件中的缺陷
 - ✓ 充当客户的技术代表，帮助客户发现软件中存在的各类问题，通过测试来演练客户验收

内容

1. 软件测试概述

✓ 软件测试的思想和原理

2. 软件测试的过程和策略

✓ 软件测试的活动及实施的方法

3. 软件测试技术

✓ 白盒和黑盒测试技术

4. 软件测试计划及输出

✓ 测试计划制定及测试结果

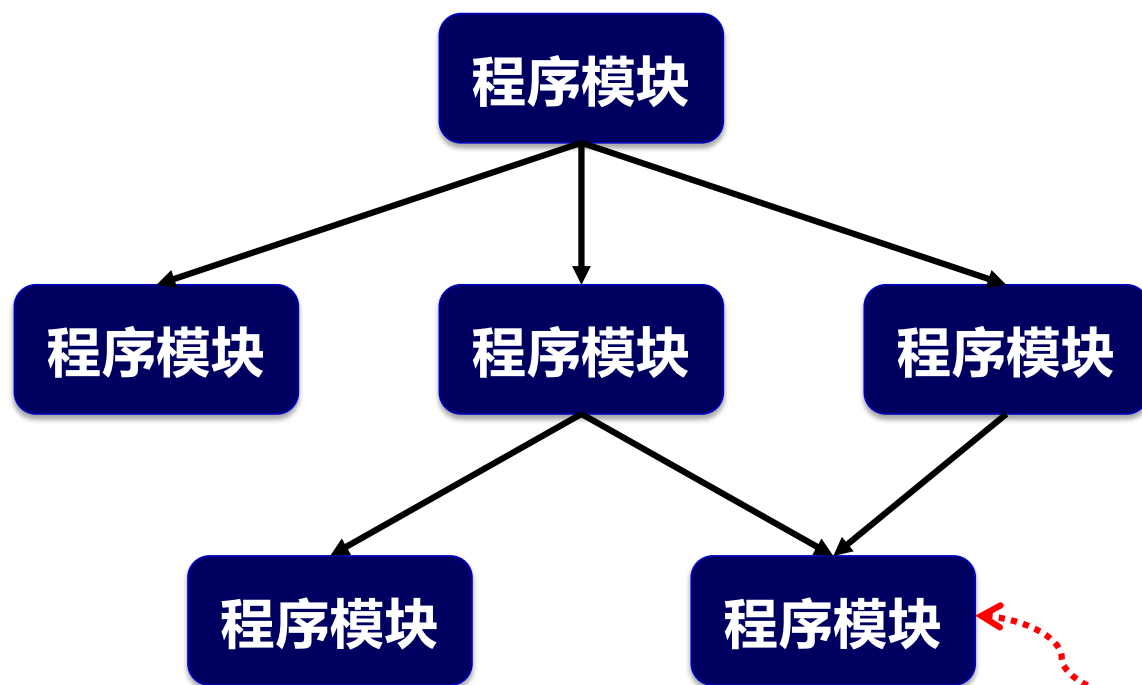


思考和讨论

□ 软件缺陷会“潜伏”在程序代码的哪些地方？



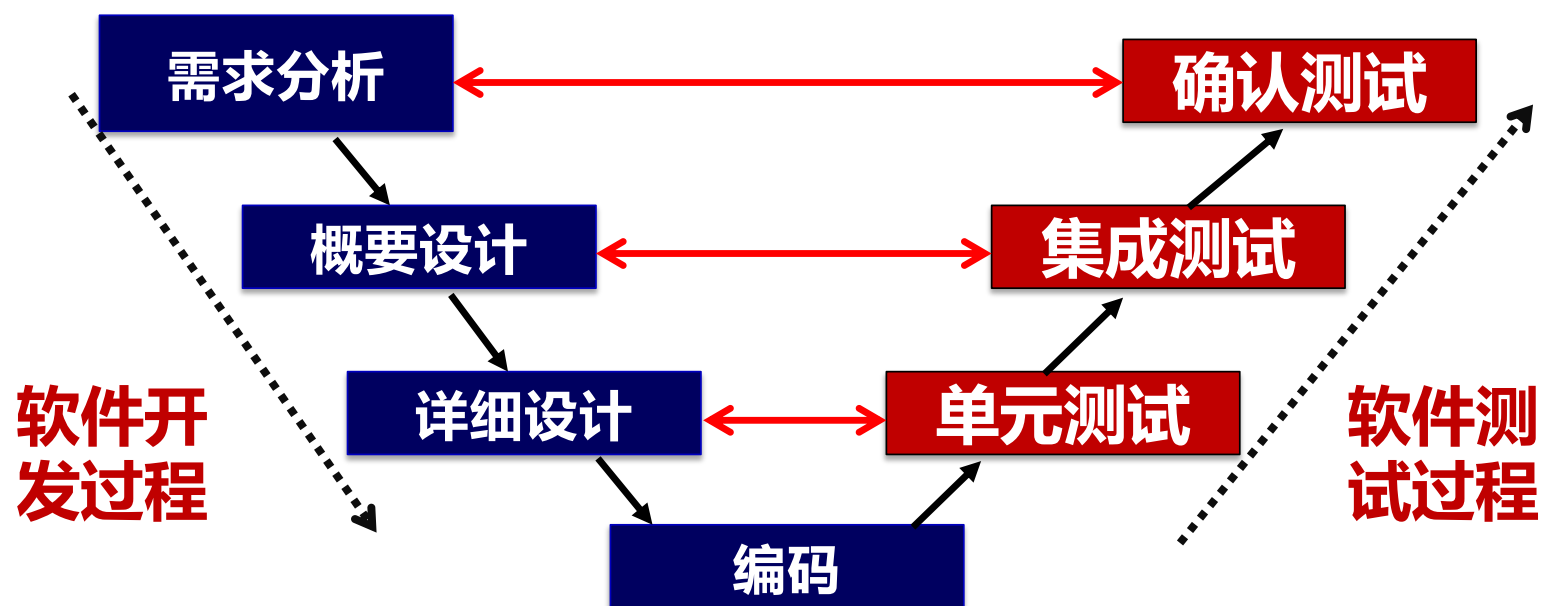
2.1 程序构成及其缺陷的潜在位置



- 程序模块内部
- 程序模块接口
- 程序模块间的交互
- 整个软件系统

发现和消除基本
模块单元中缺陷

2.2 软件测试活动



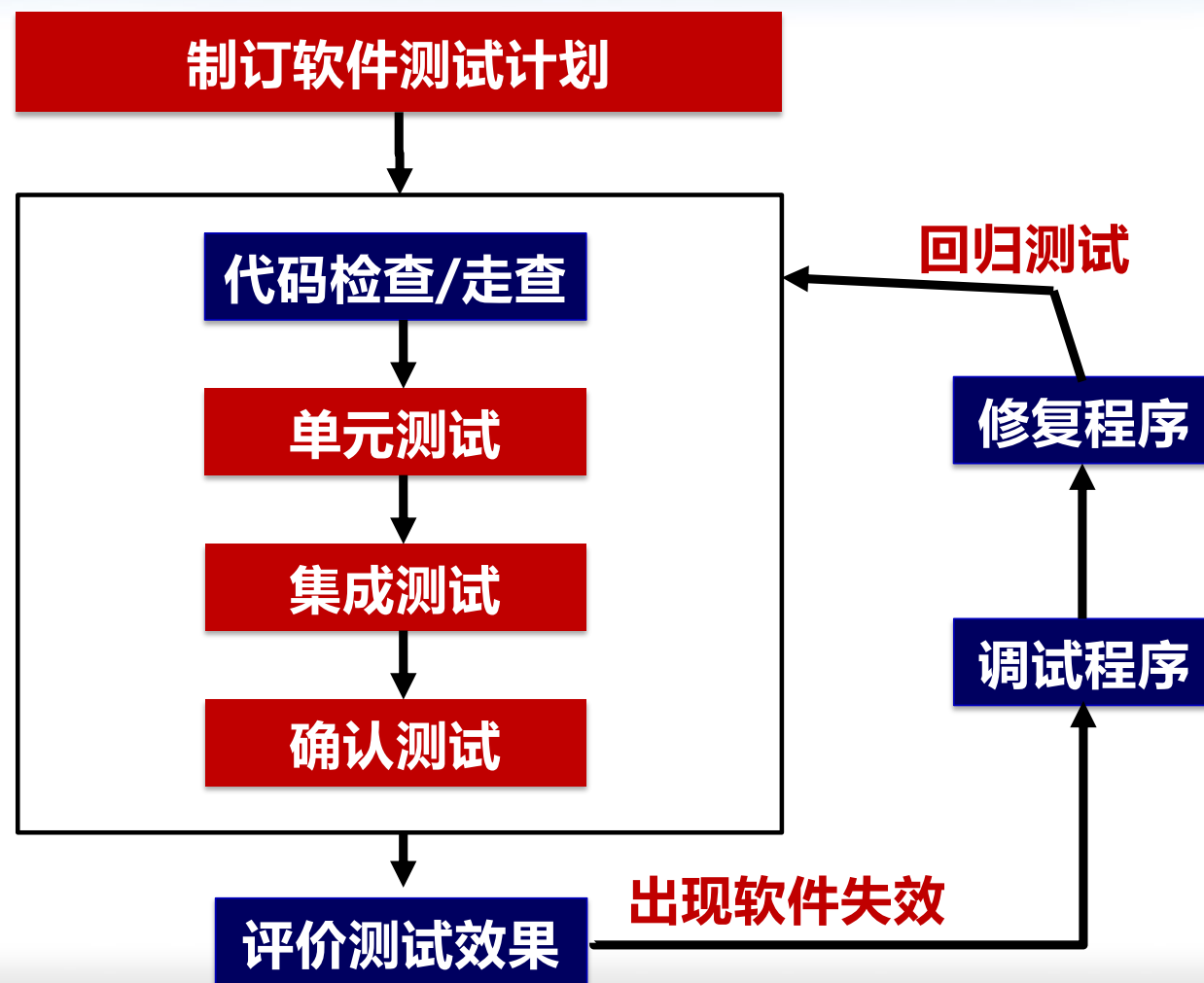
要对软件进行系统性、全方位的测试

2.3 软件测试活动之间的关系

循序渐进、逐层递进地开展软件测试



2.4 软件测试过程



1. 单元测试

□测试对象

- ✓对软件**基本模块单元**进行测试
- ✓过程、函数、方法、类

□测试方法

- ✓大多采用**白盒测试技术**

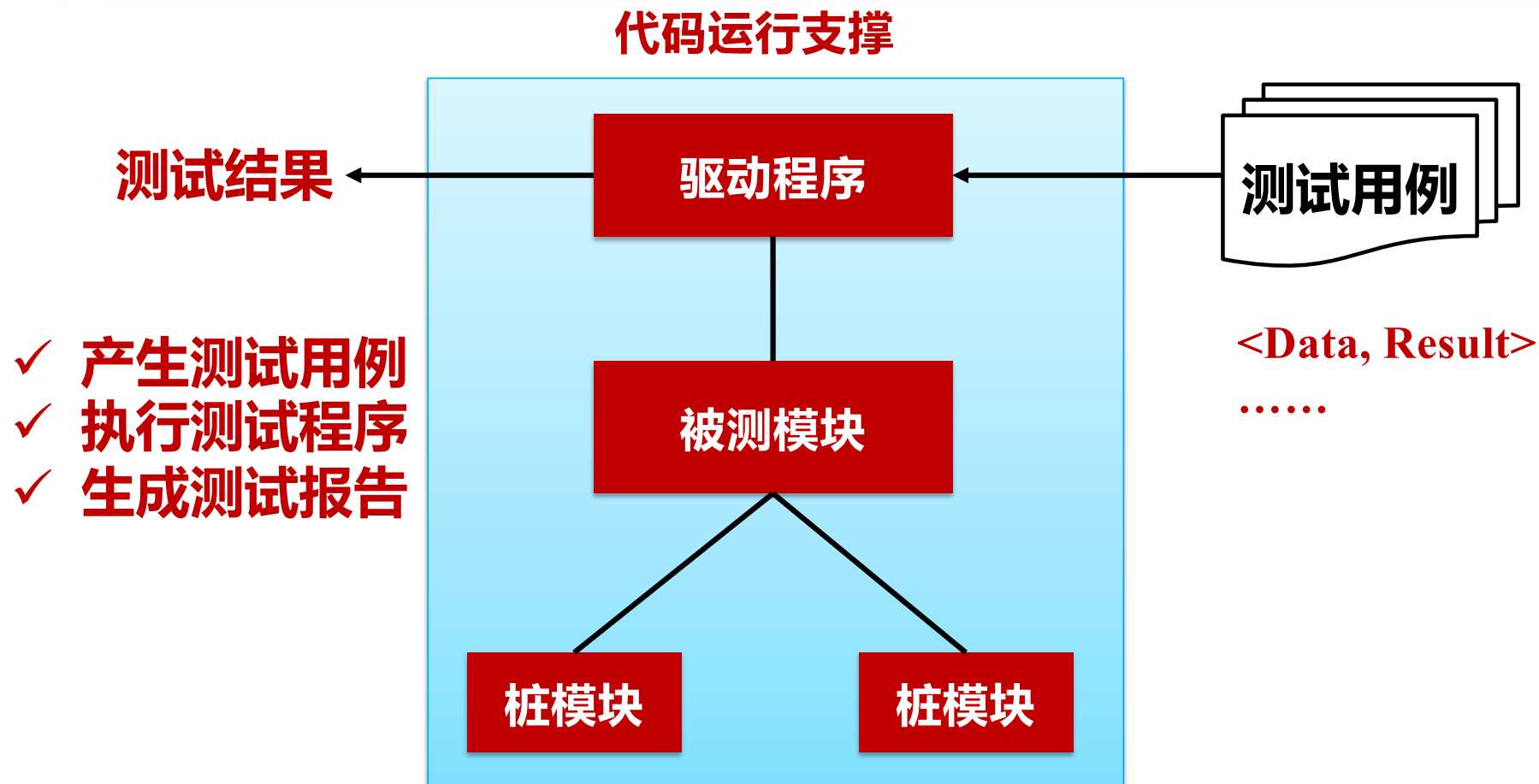
□测试的内容

- ✓模块接口测试
- ✓模块局部数据结构测试
- ✓模块独立执行路径测试
- ✓模块错误处理通道测试
- ✓模块边界条件测试



在软件详细设计阶段就需要设计单元测试用例及制定单元测试计划

单元测试的运行环境



2. 集成测试

□测试对象

- ✓对软件模块之间的**接口**进行测试
- ✓**过程调用、函数调用、消息传递、远程过程调用**

□测试技术

- ✓采用**黑盒测试技术**

□集成测试的内容

- ✓过程调用
- ✓函数调用
- ✓消息传递
- ✓远程过程调用
- ✓网络消息



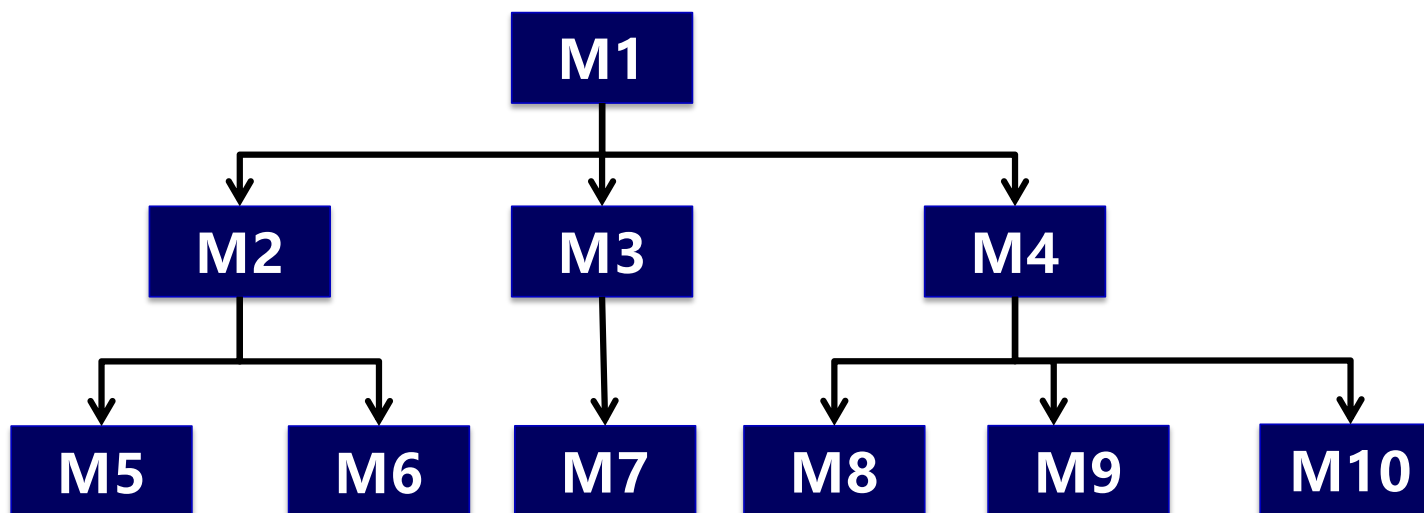
集成测试方法

□ 自顶向下集成

✓ **深度优先**或者**广度优先**的策略把各个模块进行组装和测试

□ 自底向上集成

✓ **自底向上**进行组装和测试



在概要设计阶段就需要设计集成测试用例及制定集成测试计划

自顶向下集成测试的过程

- 以**软件主控模块**作为测试驱动模块，把对主控模块进行单元测试时引入的所有**桩模块**用实际模块替代
- 依据集成策略(**深度优先或广度优先**)，每次只替代一个桩模块
 - ✓ 每集成一个模块立即测试一遍
 - ✓ 只有每组测试完成后，才着手替换下一个桩模块
- 为避免修改引入新错误，须不断进行**回归测试**（即全部或部分地重复已做过的测试）
- 从步骤（2）开始循环执行上述步骤，直至整个程序结构集成完毕

自底向上集成测试的过程

- 把低层模块集成起来形成实现某个子功能的**模块群** (cluster)
- 开发一个**测试驱动模块**，控制测试数据的输入和测试结果的输出
 - ✓对每个模块群进行测试
 - ✓删除测试使用的驱动模块，用实际开发的高层模块代替并形成能完成更大功能的新模块群
- 从步骤（1）开始循环执行上述各步骤，直至整个程序集成完毕

2.4.3 确认测试

□测试对象

- ✓对软件的**功能和性能**进行测试
- ✓判断**目标软件系统**是否满足用户需求

□依据和标准

- ✓软件需求规格说明书

□测试技术

- ✓采用**黑盒测试技术**



在需求分析阶段就需要设计确认测试用例及制定确认测试计划

α 测试和 β 测试

□ α 测试

- ✓ **软件开发公司组织内部人员**模拟各类用户行为对即将面市的软件产品（称为 α 版本、内部测试版）进行测试，发现错误并修正
- ✓ 尽可能逼真地**模拟实际运行环境和用户**对软件产品的操作，并尽最大努力涵盖所有可能的用户操作方式
- ✓ 经 α 测试并进行修改后的软件产品称为 **β 版本**（也称外部测试版）

□ β 测试

- ✓ 软件开发公司组织各方面的**典型用户**在日常工作中实际使用 β 版本，或为对外进行宣传而将 β 版本免费赠送给典型用户（很多情况下， β 版本可以通过Internet免费下载，也可以向软件公司索取），并要求用户报告异常情况、提出批评意见
- ✓ β 测试是在与开发者无法控制的环境下进行的**软件现场应用**

4. 面向对象软件测试

□ OO软件的构成基础与传统结构化程序设计不同

- ✓ OO软件中**数据和方法**的隐藏与封装
- ✓ OO编程中存在**继承、多态**等多种机制
- ✓ OO开发过程和分析、设计的重点有所不同，并且关注于**对象的交互与集成**

□ 多视点的分析和设计模型、以及与代码之间存在的映射，使得尽早测试成为可能

需要专门针对面向对象软件测试的方法

类测试的前提

□需要开发**测试驱动程序**

- ✓创建类实例、运行测试用例、向类实例（对象）发送消息
- ✓根据响应的值、对象状态来判断是否有缺陷

□类方法的测试

- ✓采用**白盒测试方法**

类方法测试的过程

1. 设置测试消息的参数、全局变量为希望的取值
2. 设置被测对象为希望的状态（设置属性值）
3. 从测试驱动程序向被测对象实例发送测试消息
4. 测试检查
 - （1）把被测方法返回值与期望值进行比较，若不同记录未通过
 - （2）对被测对象的状态期望的进行比较，若不同记录未通过
 - （3）捕获所有异常，假如是期望的异常，确定它是否正确。假如无异常或有一个不正确的异常，记录未通过
 - （4）假如必要，对全局变量的结果与期望的值进行比较，若不同记录未通过
5. 假如发生任何未通过，则诊断和修正该问题

交互测试的前提

□测试驱动程序

- ✓实例化参与交互的类实例
- ✓发送消息序列

□判断结果

- ✓交互完成后的输出
- ✓交互过程中各个对象的状态

□注意事项

- ✓消息的内容、消息的次序
- ✓正常的消息序列和不正常的消息序列

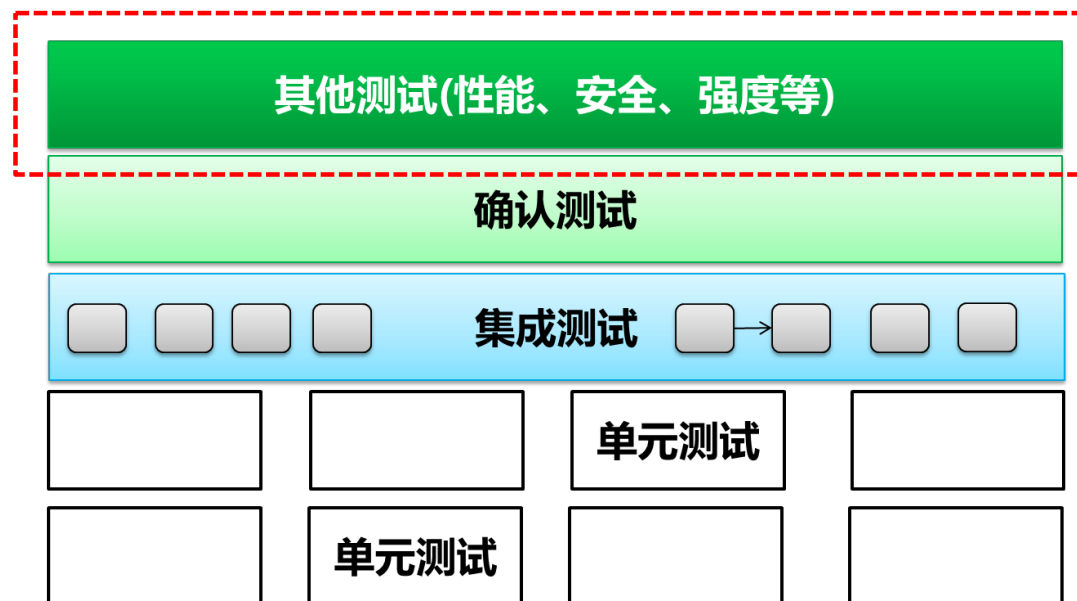
继承的测试

- 可以先测试父类，再测试子类
- 重用父类的测试用例来进行子类的测试

2.4.5 非功能性测试

- 性能测试
- 强度测试
- 配置和兼容性测试
- 安全性测试
- 可靠性测试
- 用户界面测试
- 本地化测试
- Web测试
- 安装测试
-

对于人机物三元融合系统而言，非功能测试非常重要



在需求分析阶段就需要设计非功能测试用例及制定非功能测试计划

性能测试用例示例

用例标识:	12306-NSF-1
测试项:	12306软件的性能能否支持同时有10000个用户登陆使用。并且平均响应时间少于1秒。
测试输入:	由模拟软件模拟10000个用户使用系统。
前提条件:	12306 APP和服务器软件已经完成并且安装成功。
环境要求:	使用n台服务器，配置为××××； 使用20台微机作为客户机，配置为××××； 模拟软件在客户机安装；网络通畅。
测试步骤:	(1)启动服务器系统； (2)由模拟软件在每个客户机分别依次发送50个用户登陆请求，并模拟查询车次的过程； (3)监控程序把每个用户的请求和服务器响应过程、时间记录在日志中； (4)对记录结果进行处理，判断是否每个用户均获得成功的服务，并计算平均响应时间。
预期输出:	每个模拟用户都得到正确服务，且对请求的平均响应时间小于1秒。

强度测试

□强度测试可被视为性能测试一部分

- ✓ 性能测试是为测试软件系统在正常使用时是否能够达到一定性能指标
- ✓ 强度测试则是对系统不断施加更大的**压力和使用强度**，确定系统瓶颈或者不能接受性能点，来获得系统能提供最大服务能力

□强度测试示例

- ✓ 12306系统同时500个用户使用情况下正常运行属于性能测试
- ✓ 把测试场景定为上千用户、甚至上万用户同时使用，那么就是一种强度测试了，**以判断系统能够承受最大的强度是多少**

□强度测试适用于在可变负载系统中运行的软件

- ✓ 强度测试可与性能测试一起进行，但需对测试用例中定义的测试环境、条件、输入、步骤等内容进行调整，以适应强度测试

安全性测试

□设计测试用例来暴露软件安全漏洞的过程

- ✓如设法破坏数据库管理系统的数据安全机制、突破重要领域软件系统的用户访问控制等

□安全性测试的过程

- ✓明确潜在的**安全隐患**，包括容易遭受攻击的内容、可能作为潜在入侵者的人员角色等
- ✓明确**潜在的入侵行为及时机**，例如事务初始化、系统输入、执行存储和检索操作等时候
- ✓列出每种潜在安全隐患可能遭到的**入侵行为及其可能性**
- ✓确定**风险较高的入侵点**，这可通过对包括发生可能性、后果严重程度等因素进行分析得到
- ✓根据风险的顺序，设计测试用例来实施安全性测试

用户界面测试

□也称**易用性测试**，测试用户界面是否**易于使用**且具有较强**实用性**

- ✓是否把常用的**按钮或菜单项**放在醒目的位置
- ✓软件的输出是否**清晰、有序**
- ✓错误信息是否**直观和易于理解**
- ✓软件**错误顺序**是否符合用户的业务过程
- ✓界面的**风格**是否一致

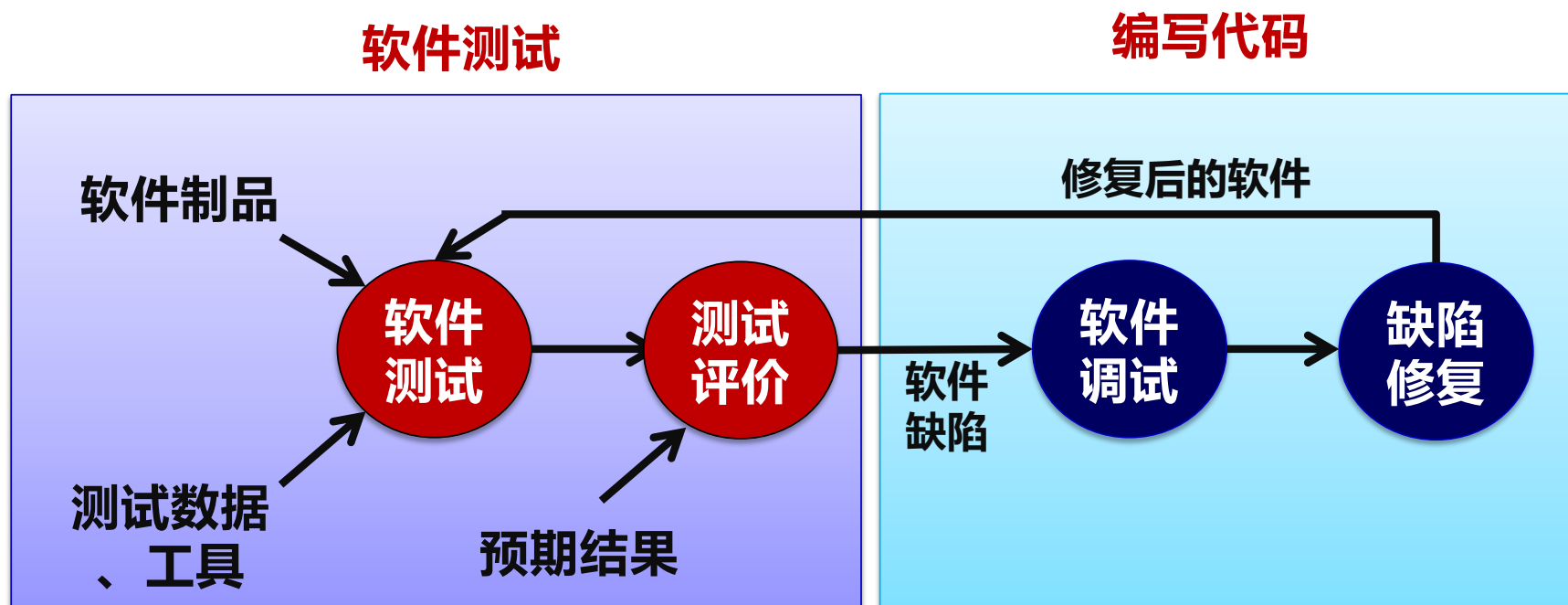
Web测试

□基于Web软件利用浏览器通过超链接把各个网页联系在一起，满足用户对信息的获取

□需要测试大量的对象

- ✓文本测试，对其中的术语、题目、内容素材、电子邮件地址、电话号码等信息的准确度、时效性方面进行检查，文字的拼写是否有误也要经常进行检查
- ✓由于窗口缩放会引起文字段落改变，应测试是否会导致格式混乱
- ✓测试超级链接
- ✓测试图片和视频
- ✓测试表单

2.5 软件测试的后续工作



测试是为了发现错误，定位缺陷和纠正错误不属于软件测试的工作范畴

回归测试

□修改程序可能会引入新的错误

- ✓原先“正常”的程序现在变得“不正常”

□回归测试目的

- ✓验证软件新版本是否从正常状态回归/退化到不正常状态

□方法

- ✓重新进行测试，再次运行所有的测试用例来发现缺陷

□回归测试最好能够自动化

- ✓单元测试是回归测试的基础

测试、调试和排错

□目的

- ✓测试发现缺陷
- ✓调试定位缺陷
- ✓排错纠正错误

□独立性不同

- ✓测试由独立的测试小组进行
- ✓调试和排错由开发人员完成

测试的重要性和特殊性

□软件测试是软件质量保证活动中关键步骤

- ✓对SRS、设计规格说明书以及编码的最后复审
- ✓其工作量往往占软件开发总工作量的40%以上
- ✓确保软件质量的一种有效和可操作的重要手段

□软件测试有其特殊性和规律

- ✓软件是逻辑产品
- ✓软件有其复杂性
- ✓软件缺陷隐蔽性

2.6 软件测试原则(1/2)

□测试应该有计划

✓ 在开发初期就应制定测试计划，测试计划应该在测试之前就应该制定

□所有的测试都应该追溯到用户需求

□将Pareto原则应用于软件测试

✓ 80%的错误在大约20%的模块中找到根源

□测试应该从“微观”开始，逐步转向“宏观”

□穷举测试是不可能的

□每个测试用例都必须定义预期的输出或结果

□尽量避免程序的开发人员来测试他自己编写的代码

□尽量避免程序开发组织测试其自己开发的程序

软件测试原则(2/2)

- 应详细检查每次**测试的结果**
- 测试用例中不仅要说明**合法有效的输入条件**，还应该描述那些**不期望的、非法的输入条件**
- 检查一个程序没有完成希望它做的事情只是测试的一半任务，还应检查程序是否执行了**不希望它做的事情**
- 避免随意舍弃任何测试用例，即使非常简单的测试用例
- 不应在事先假设不会发现错误的情况下制定测试计划
- 一个程序模块**存在更多错误的可能性与该模块已经发现的错误数量成正比**
- 测试是一个具有相当**创新性和智力挑战**的活动

内容

1. 软件测试概述

✓ 软件测试的思想和原理

2. 软件测试的过程和策略

✓ 软件测试的活动及实施的方法

3. 软件测试技术

✓ 白盒和黑盒测试技术

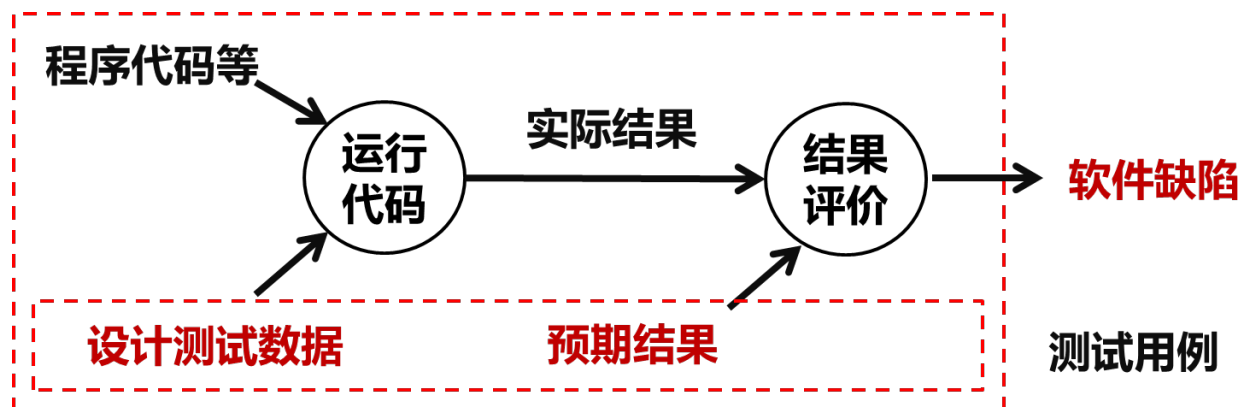
4. 软件测试计划及输出

✓ 测试计划制定及测试结果



思考和讨论

- 如何来**设计测试用例**？
- 要设计**多少个测试用例**？
- 如何才能做到**充分的软件测试**？即通过设计足够多的测试用例来发现尽可能多的软件缺陷



3.1 程序单元测试

□测试对象-程序单元

- ✓基本模块单元
- ✓基本类方法、函数和过程

□测试内容-四个方面

- ✓执行路径
- ✓错误处理
- ✓模块接口
- ✓边界条件

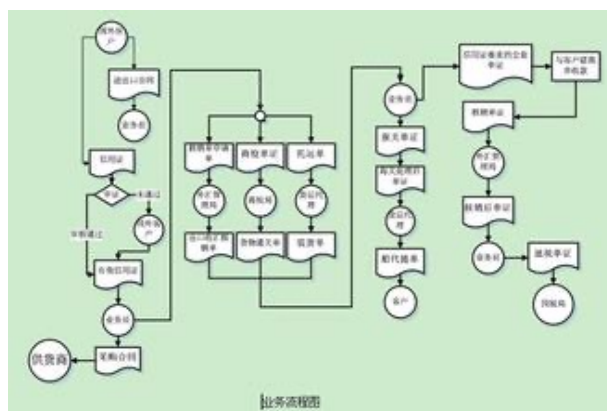
□测试依据-设计文档

- ✓程序流程

程序单元是构成软件系统的基本要素，必须对其进行测试以发现缺陷

程序单元测试的依据和对象

测试依据：设计文档



根据设计文档
编写程序代码

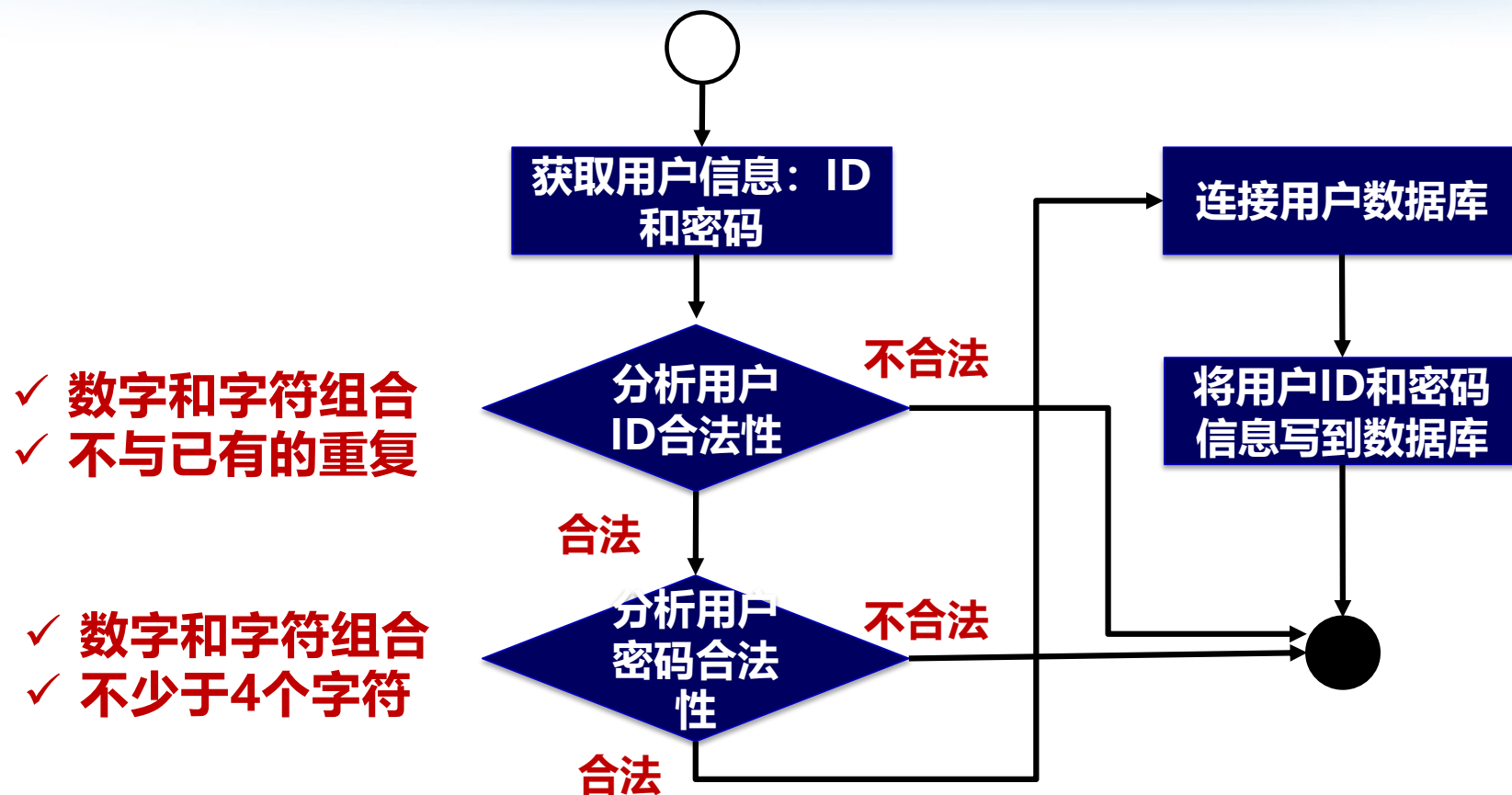
```
1 package MarDetector;
2
3 import java.awt.Image;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21 /* 构造类：基类机器人，属性：当前位置，方法：在网络上由当前位置移动到另外一个位置 */
22 public class BasicRobot extends Agent {
23     /* 基类机器人的位置 */
24     public Coordinate posi = null;
25     private BasicRobot basicrobot = null;
26
27     public BasicRobot(){
28         basicrobot = new BasicRobot();
29     }
30     /* 设置机器人的位置 */
31     public void setRobotPosi(Coordinate p){
32         posi = p;
33     }
34
35     /* 获取机器人的位置 */
36     public Coordinate getRobotPosi(BasicRobot thisrobot){
37         return(thisrobot.posi);
38     }
39     /* 基类机器人由当前位置移动到另外一个位置方法实现 */
40     public void moveStep(Coordinate start, Coordinate next) {
41         ShortestPath path = new ShortestPath();
42         path.setStart(start.x, start.y);
43         path.setTarget(next.x, next.y);
44     }
45
46
47     protected void setup() {
48         System.out.println("BasicRobot name is " + getLocalName());
49         addBehaviour(new RandomWalkBeh(this));
50     }
51
52
53
54 }
```

测试对象：模块代码

根据设计信息来设计测试用例

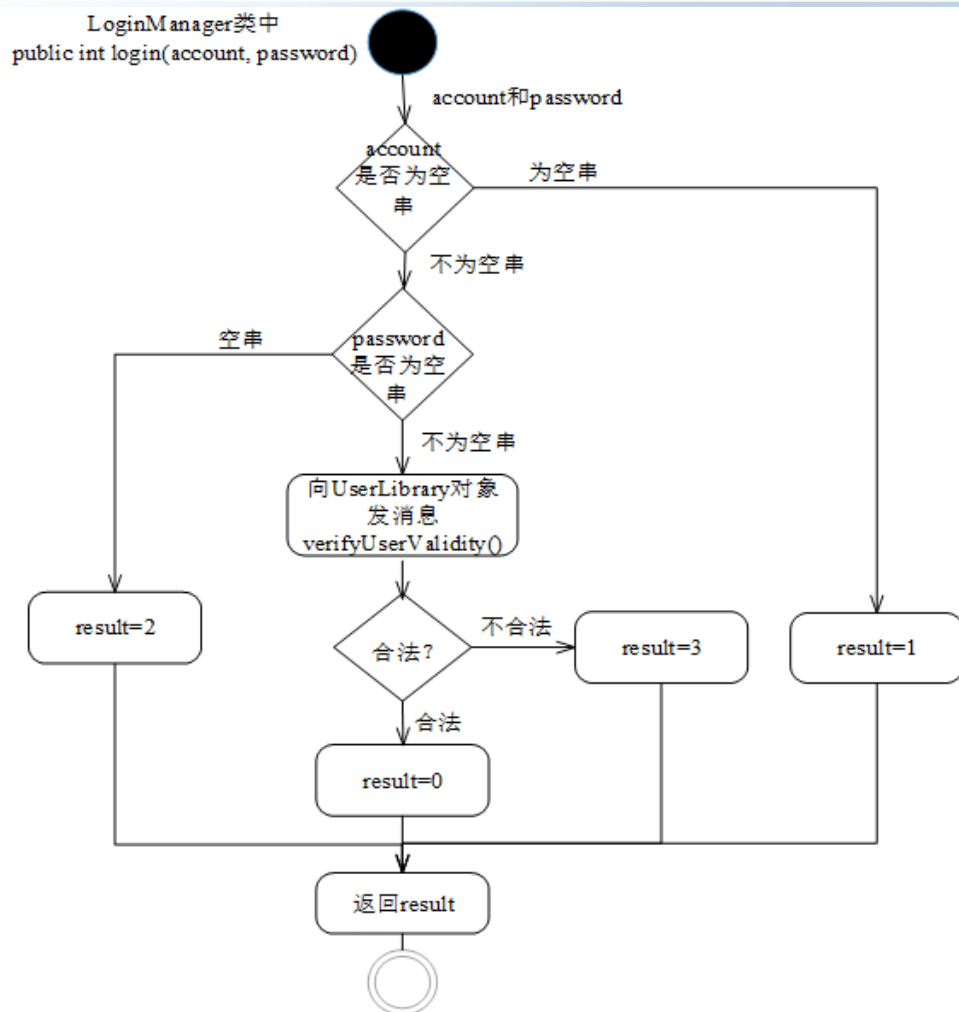
根据测试用例对代码进行测试

示例：程序单元的设计描述-用户注册



软件设计的流程图描述

示例：程序单元的设计描述-用户登录



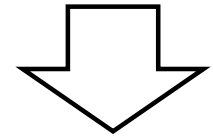
软件详细设计信息的
流程图描述

3.2 程序单元测试需要解决的问题

□如何设计测试用例？

- ✓根据设计文档设计测试用例
- ✓设计多少个测试用例
- ✓测试用例的有效性：发现问题
- ✓测试用例的简练性：不冗余，造成重复测试

设计程序单元的
测试用例



□如何运行程序单元来发现缺陷？

- ✓程序单元的运行
- ✓程序单元对测试数据的处理
- ✓根据什么来发现缺陷

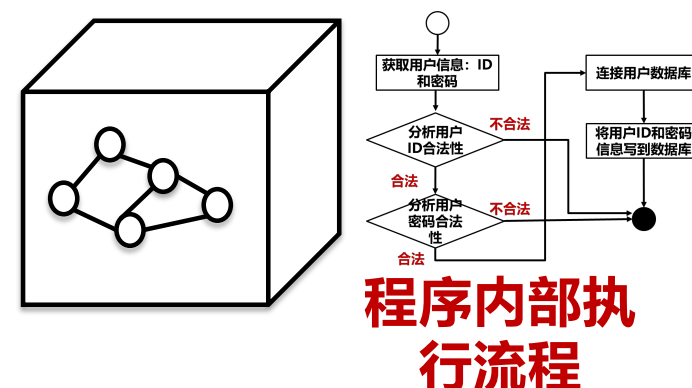
运行程序单元和
测试用例以发现
缺陷

3.3 软件测试技术

□如何设计和运行测试用例

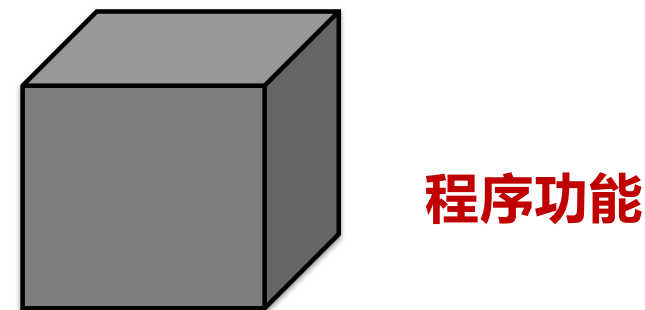
□白盒测试技术

✓基于程序内部的执行流程来设计测试用例



□黑盒测试技术

✓基于程序的外在功能和接口来设计测试用例



盒子对应于基本模块单元（如类方法、函数、过程等），测试基本对象

3.4 软件测试技术-白盒测试

□设计测试用例思想

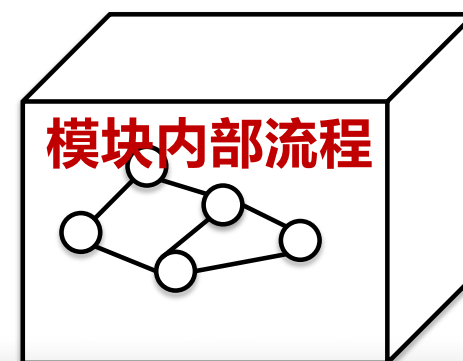
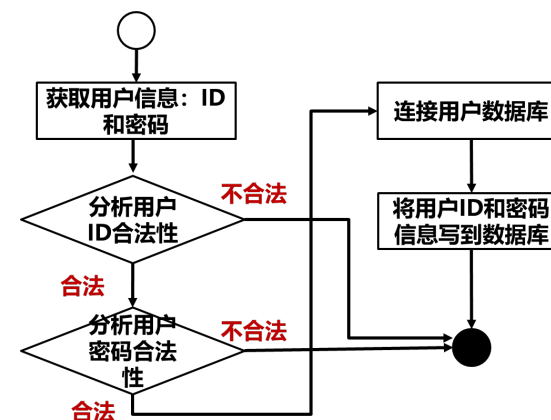
- ✓根据**程序单元内部工作流程**来设计测试用例

□发现程序单元缺陷

- ✓运行待测试的程序，**检验程序是否按内部工作流程来运行的**，如果不是则存在缺陷

□特点

- ✓必须了解程序的**内部工作流程**才能设计测试用例



白盒测试用例设计的指导原则

□如何设计测试用例?

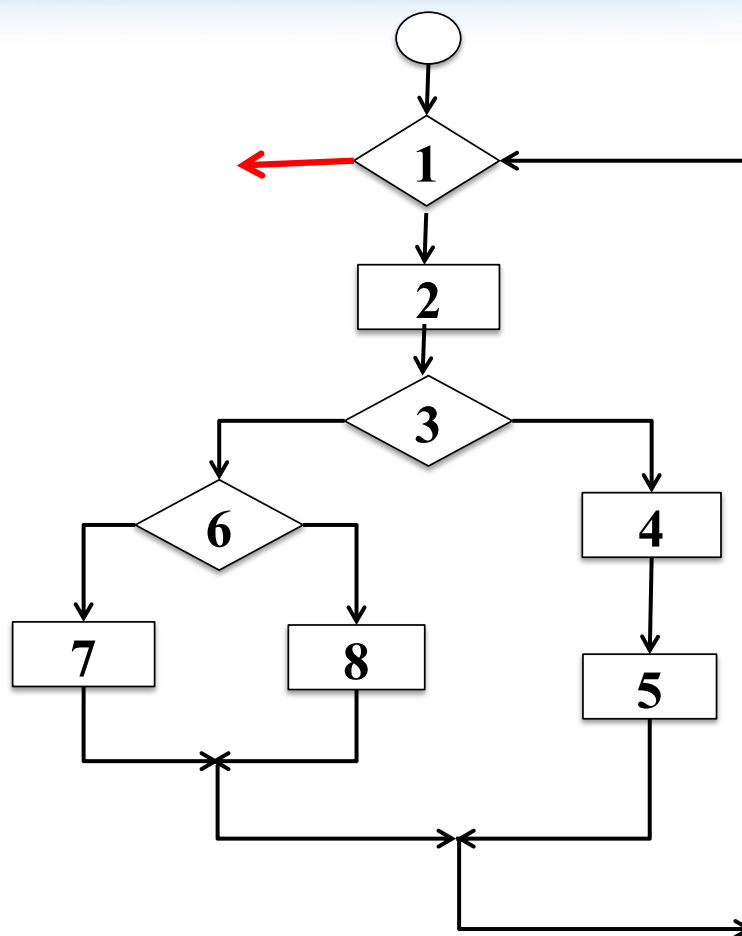
- ✓内部执行流程
- ✓生成测试数据

□设计多少测试用例?

- ✓遵循覆盖原则

□测试用例覆盖准则

- ✓语句覆盖
- ✓分支覆盖
- ✓路径覆盖
- ✓基本路径覆盖



- ✓语句覆盖
- ✓分支覆盖
- ✓路径覆盖
- ✓基本路径覆盖

程序的路径可能有无穷条，但是基本路径只有有限条

基本路径测试的思想

□思想

✓ 路径 → 基本路径 → **基本路径测试**

□基本路径

✓ **至少引入一个新语句或者新判断**的程序通道

□前提

- ✓ 软件模块的逻辑结构（流程图）
- ✓ 据此可掌握模块的基本路径

□如何设计测试用例实现基本路径覆盖

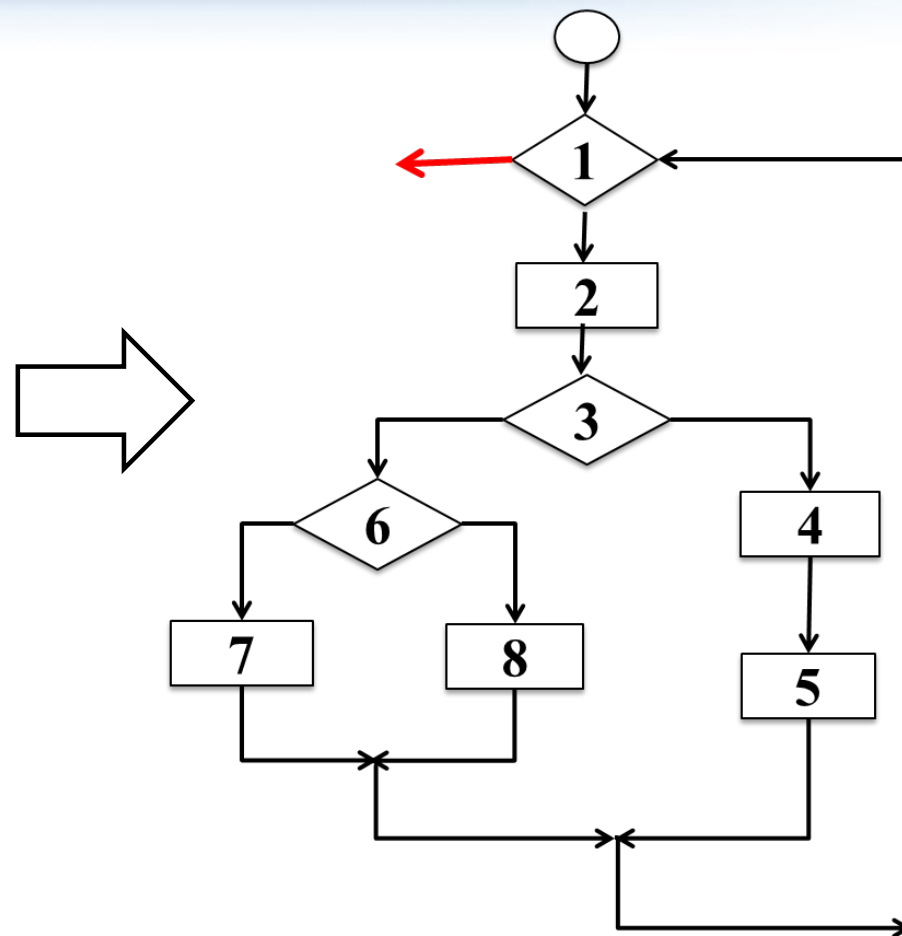
✓ 哪些基本路径 → **流图** → **流程图**

**基本路径测试可实现
基本路径的测试覆盖**

步骤1: 根据程序逻辑画出流程图

```
void Func(int nPosX, int nPosY) {  
    while (nPosX > 0) {  
        int nSum = nPosX + nPosY;  
        if (nSum > 1) {  
            nPosX减一;  
            nPosY减一;  
        }  
        else {  
            if (nSum < -1) nPosX -= 2;  
            else nPosX减四;  
        }  
    }  
    // end of while  
}
```

软件模块设计细节

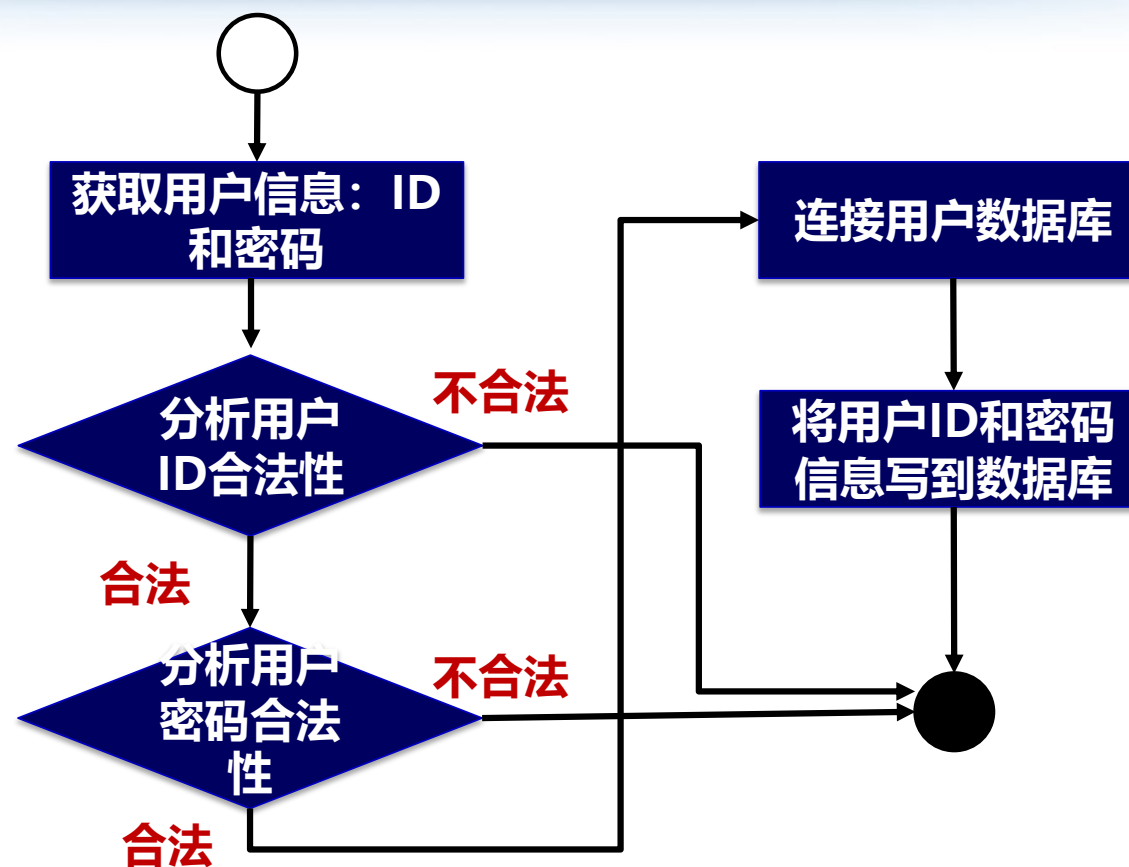


注意流程图的正确画法!

示例：程序流程图

✓ 数字和字符组合
✓ 不与已有的重复

✓ 数字和字符组合
✓ 不少于4个字符



用户注册模块的流程图

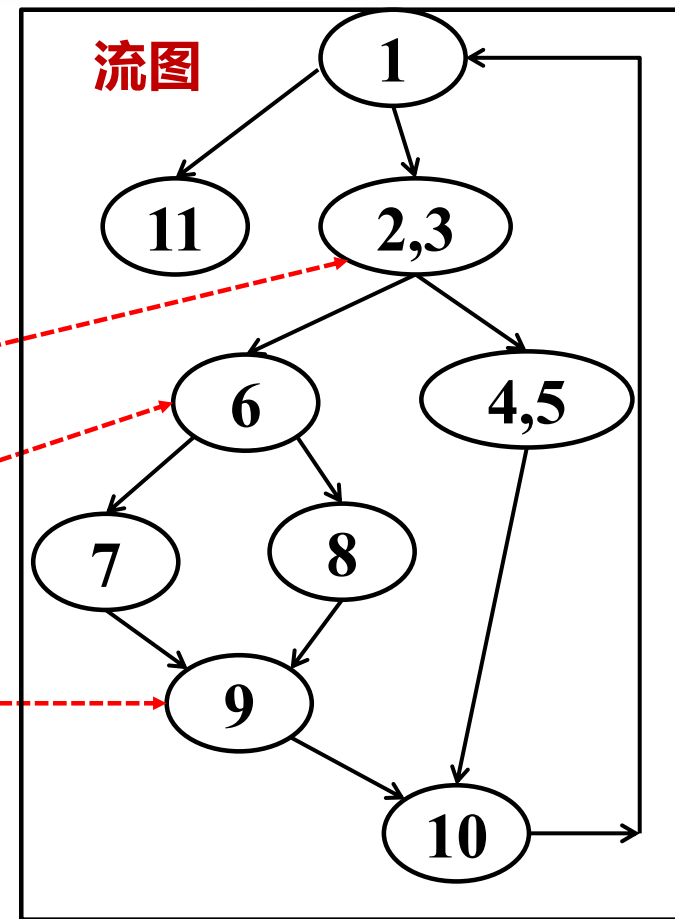
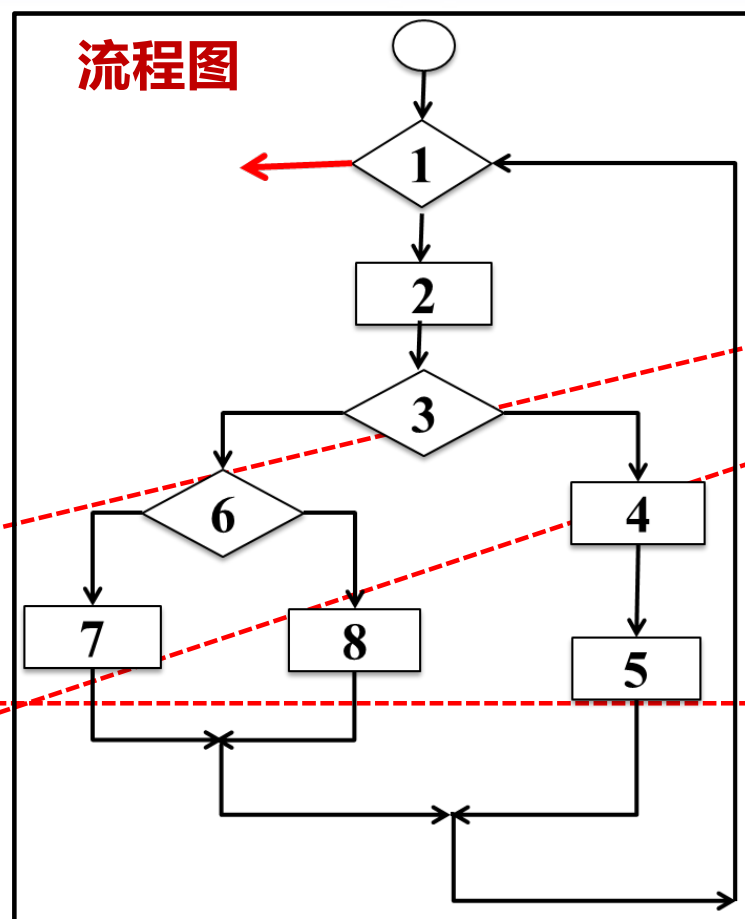
步骤2: 将流程图转换为流图 (1/2)

□何为流图

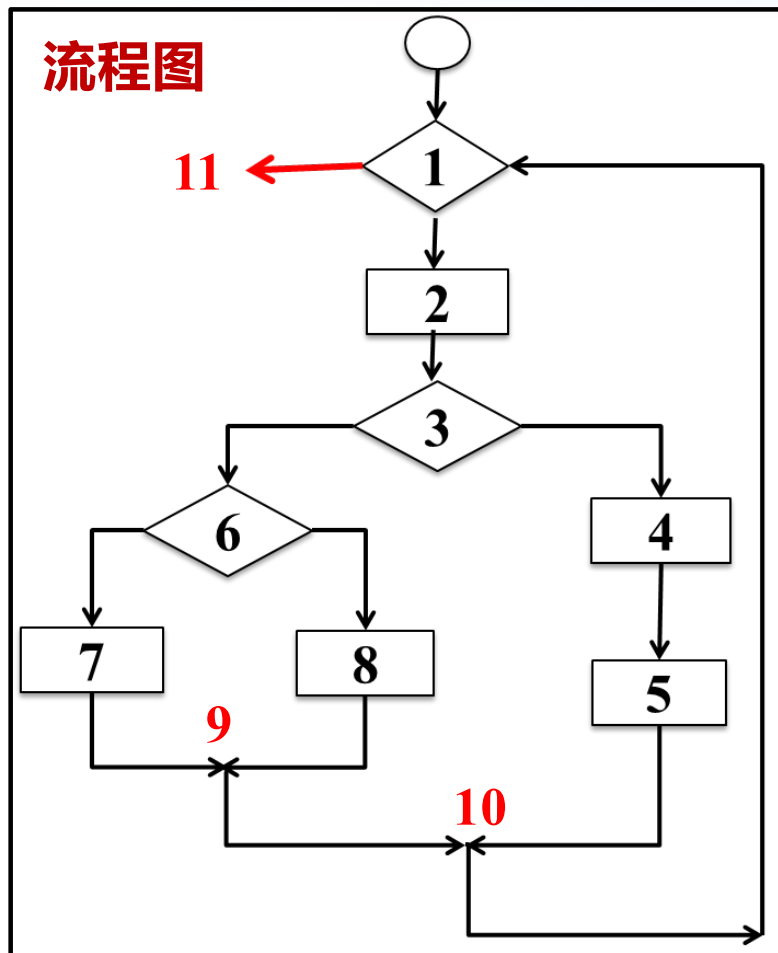
- ✓ 流图刻画程序控制结构但不涉及程序过程性细节

□节点形式

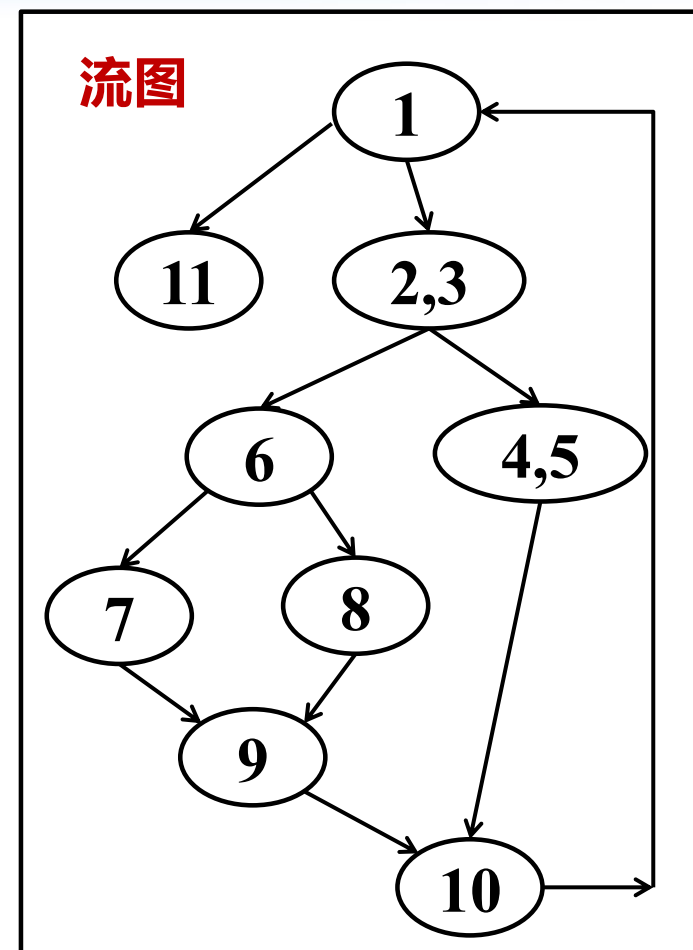
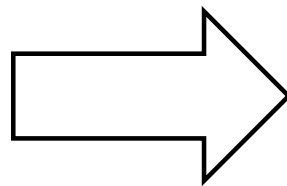
- ✓ 过程块
- ✓ 结合点
- ✓ 判定点



步骤2: 将流程图转换为流图 (2/2)



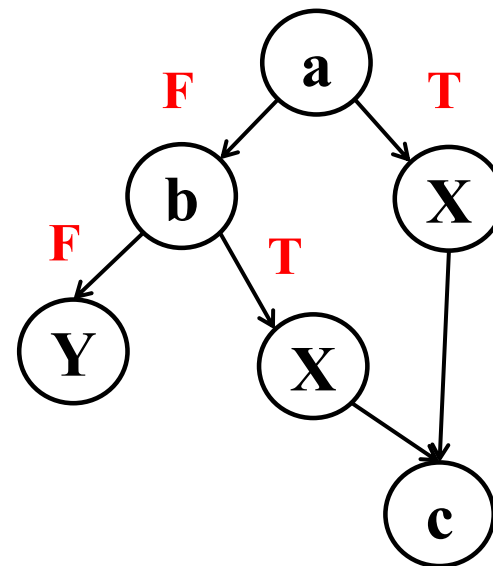
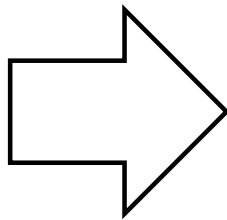
判定点
过程块
结合点



流图中的判定点不应含复合条件

□ 否则按下列方式增加判定点

If a or b
Then X
Else Y
End If



步骤3: 确定基本路径集合

□基本路径的数量

✓流图Cyclomatic复杂度

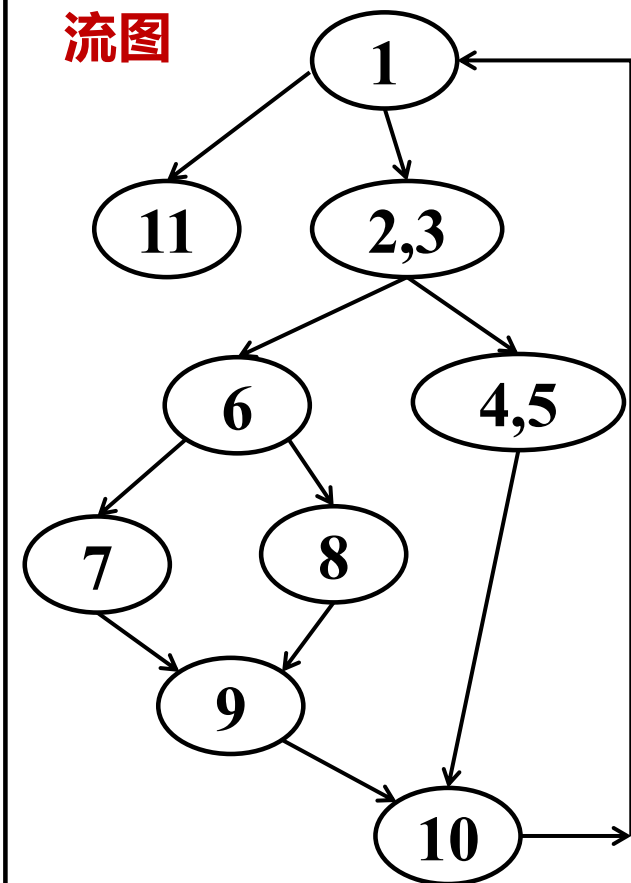
$$✓V(G) = E(\text{edges}) - N(\text{nodes}) + 2$$

$$✓V(G) = 11 - 9 + 2 = 4$$

- ① 1 - 11
- ② 1 - 2, 3 - 6 - 7 - 9 - 10 - 1 - 11
- ③ 1 - 2, 3 - 4, 5 - 10 - 1 - 11
- ④ 1 - 2, 3 - 6 - 8 - 9 - 10 - 1 - 11

4条基本路径

流图



步骤4: 根据基本路径设计测试用例

基本路径

□ 1-11

✓ nPosX 取-1, nPosY取任意值

□ 1 - 2, 3 - 4, 5 - 10 - 1 - 11

✓ nPosX 取1, nPosY取1

□ 1- 2, 3- 6-7 - 9 - 10-1- 11

✓ nPosX 取1, nPosY取-1

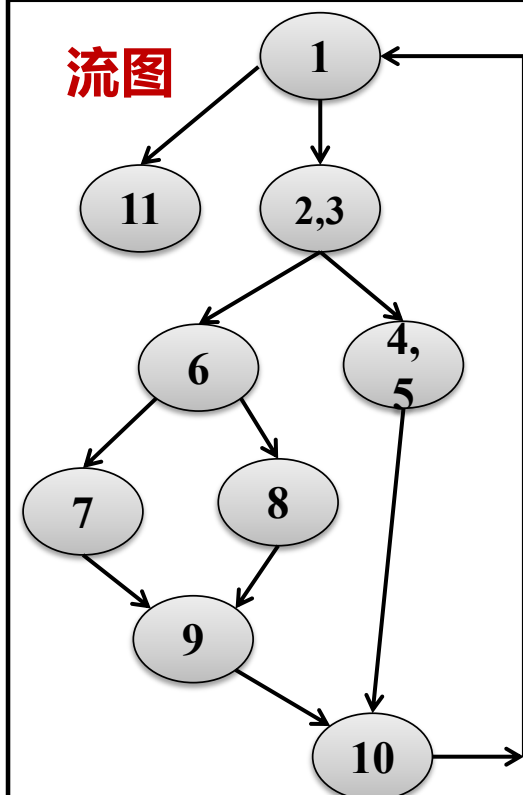
□ 1- 2, 3 - 6 - 8 - 9 - 10-1-11

✓ nPosX 取1, nPosY取-3

设计信息

```
void Func(int nPosX, int nPosY) {  
    while (nPosX > 0) {  
        int nSum = nPosX + nPosY;  
        if (nSum > 1) {  
            nPosX减一;  
            nPosY减一;  
        }  
        else {  
            if (nSum < -1) nPosX -= 2;  
            else nPosX减四;  
        }  
    }  
    // end of while  
}
```

流图



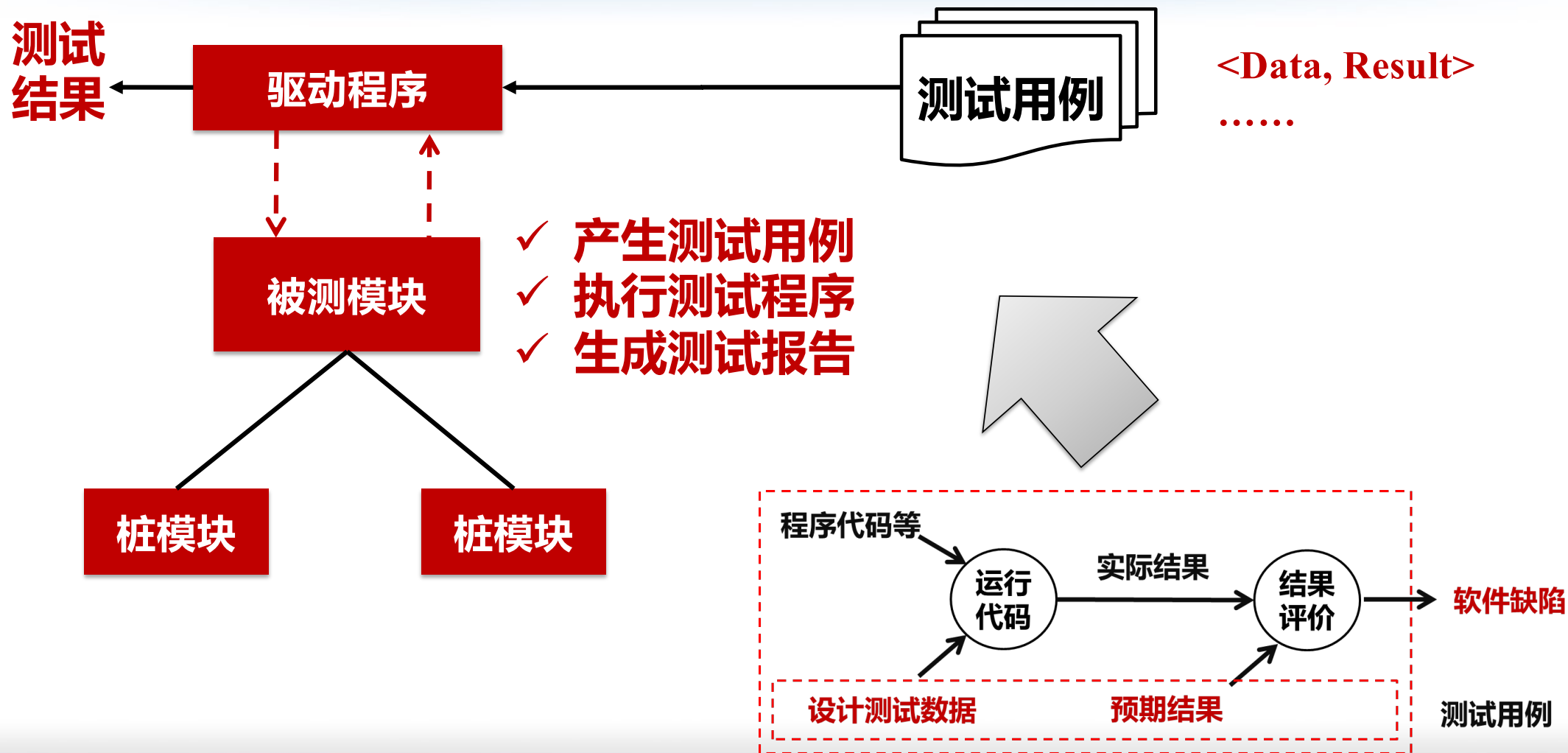
描述每一个测试用例

<u>用例标识:</u>	对该测试用例赋予一个 唯一标识
<u>用例开发者:</u>	谁 编写的本用例
<u>用例开发日期:</u>	编写用例的 日期
<u>测试项:</u>	描述将被测试的具体特征、代码模块等对象
<u>测试输入:</u>	测试时为程序提供的 输入数据
<u>前提条件:</u>	执行测试时系统应处于的 状态或要满足的条件 等
<u>环境要求:</u>	执行测试所需的 软硬件环境、测试工具、人员 等
<u>测试步骤:</u>	(1)...; (如点击“文件”菜单中的“新建”菜单项) (2)...; (如在“test case”目录下选择“test5.dat”文件)
<u>预期输出:</u>	希望程序运行得到的 结果
<u>用例间的依赖性:</u>	该测试用例依赖或受影响的其它测试用例

步骤5: 运行程序检验测试用例

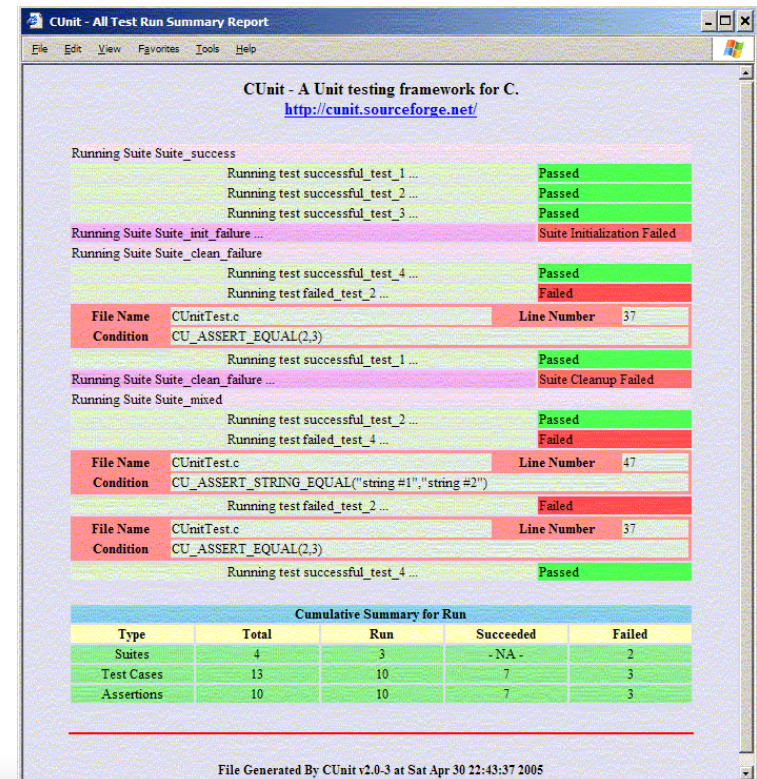
- 运行待测试的程序代码
- 逐个输入测试用例
- 分析程序的运行路径
- 如果运行路径与期望路径不一样，则存在缺陷

程序单元测试的运行环境



单元测试工具

- C/C++语言 - C++Test、Cunit、CppTest
- .Net开发 - Nunit
- Java语言 - Junit
- Python语言 - PyUnit



3.5 JUnit

- JUnit是一个Java语言的单元测试框架
- 由Kent Beck和Erich Gamma建立
- 开源软件



<http://junit.org/>

下载并安装

JUnit的提出者

□ 1997年Erich Gamma和Kent Beck为Java语言创建了一个简单有效的单元测试框架



Erich Gamma
《设计模式》作者之一

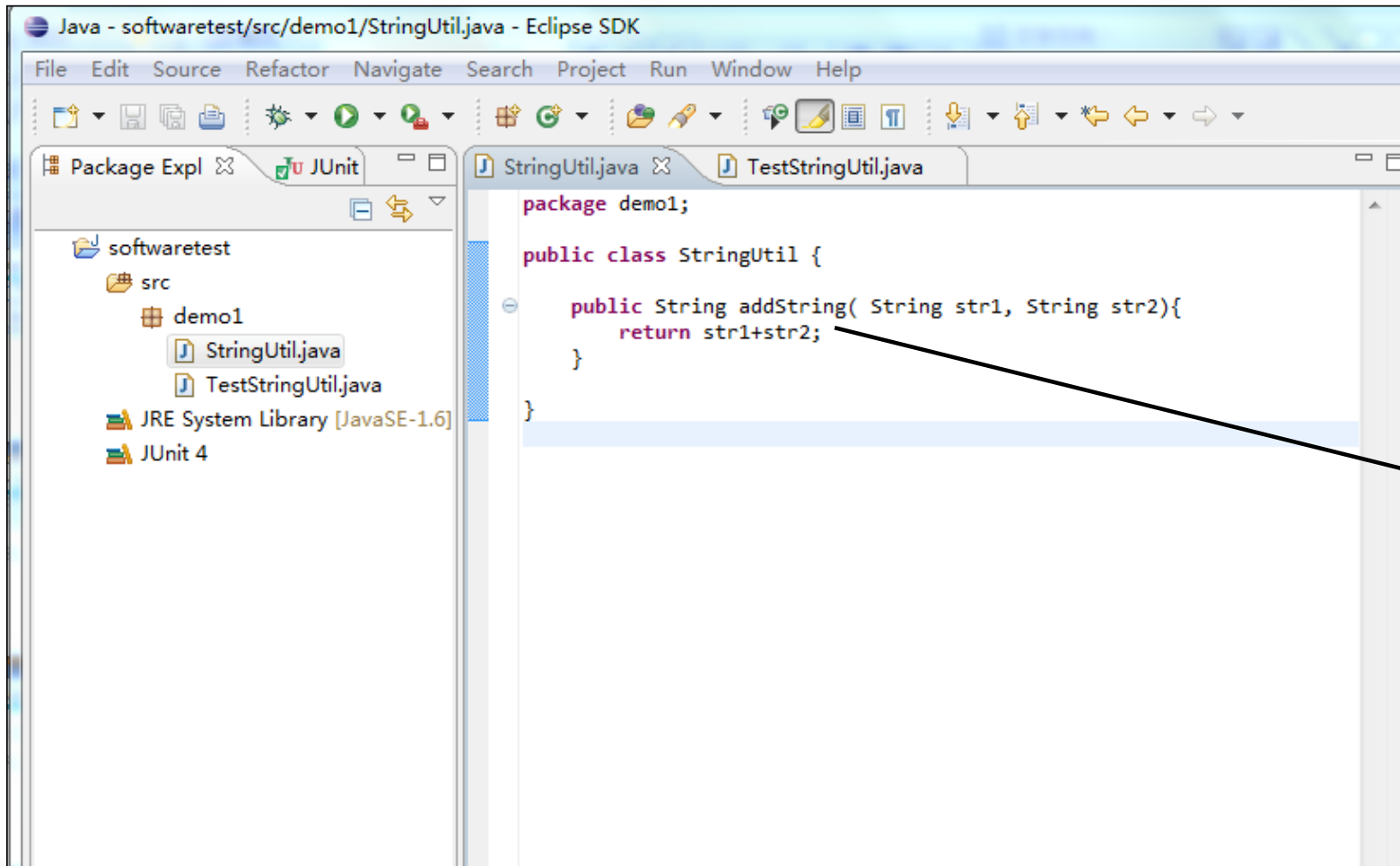


Kent Beck
提出软件开发方法“极限编程”
《重构:改善既有代码的设计》作者

在Eclipse中使用JUnit

1. 建立一个被JUnit测试的类
2. 建立对应的JUnit Test类
3. 针对自动生成的代码进行修改
4. 执行测试用例

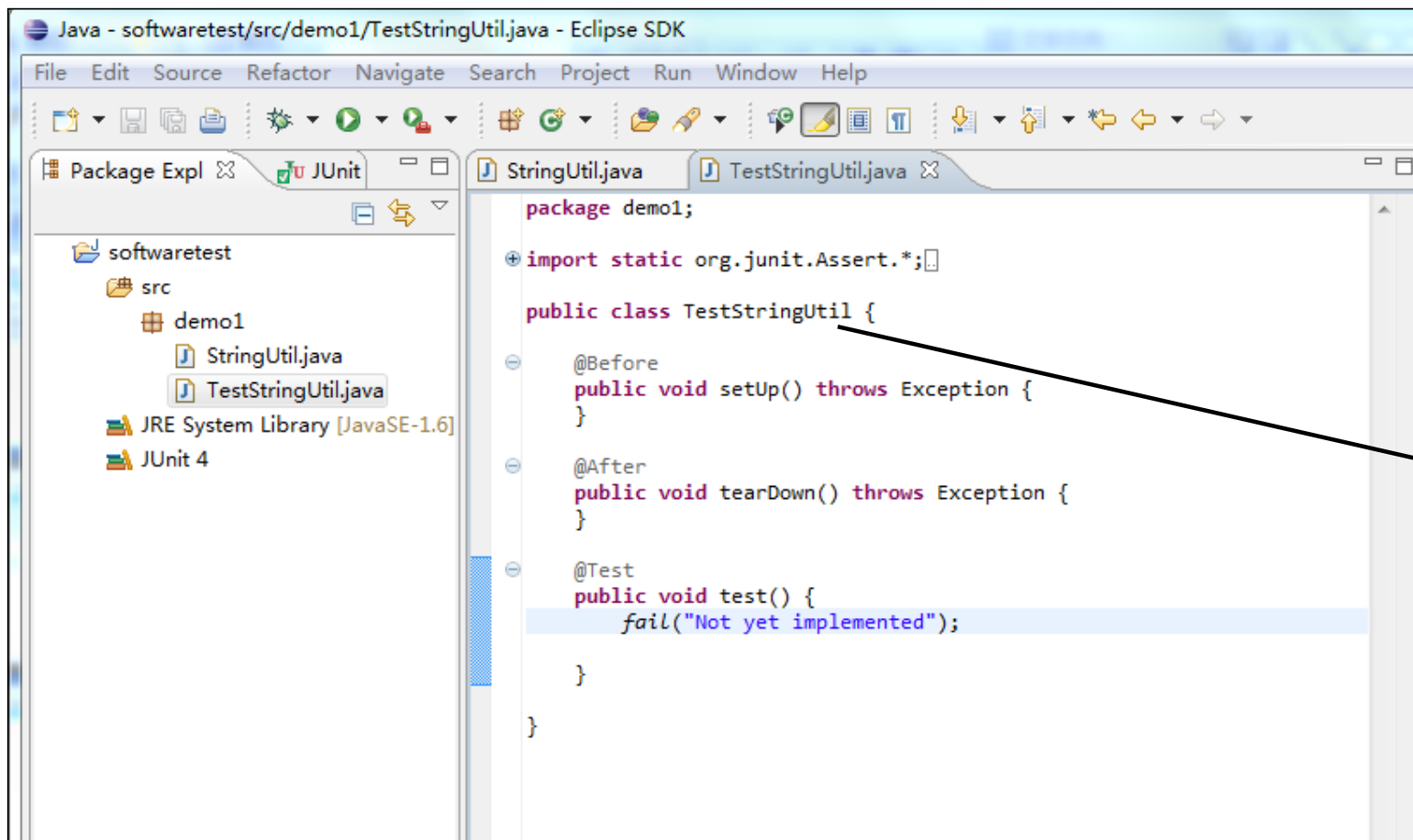
1. 建立一个被JUnit测试的类



被测试的对象

**被测试的程序单元:
类方法
addString ()**

2. 建立对应的JUnit Test类



类似于主控模块及桩模块

编写测试类

Assert方法（测试用例预期输出）

❑ **assertArrayEquals**

✓ 判断两个数组是否相等

❑ **assertEquals**

✓ 判断两个对象是否相等

❑ **assertFalse**和**assertTrue**

✓ 判断布尔变量是否为False或True

❑ **assertNotNull**和**assertNull**

✓ 判断一个对象是否为空

❑ **assertNotSame**

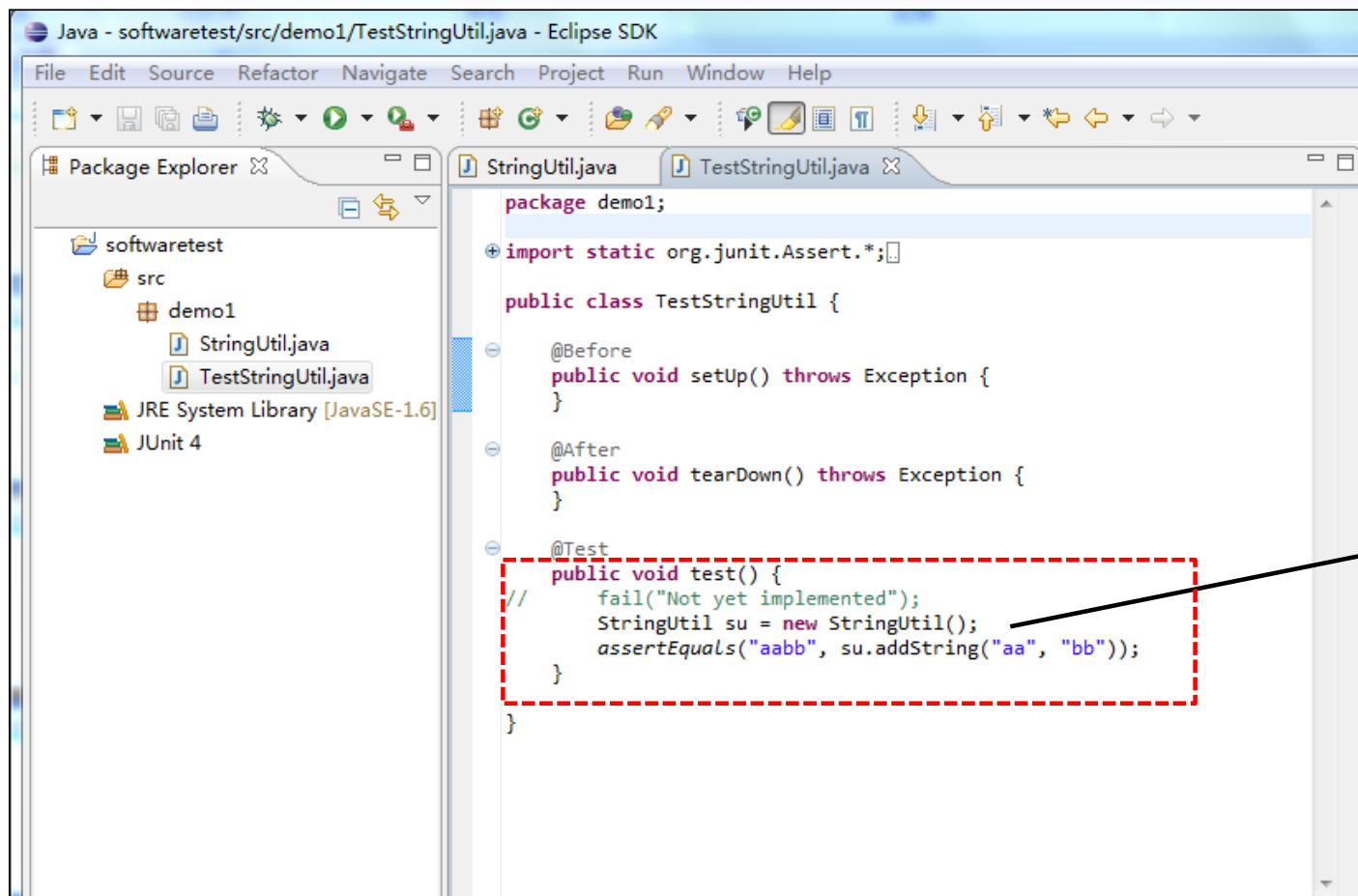
✓ 判断两个引用是否指向同一个对象

❑ **Fail**

✓ 让测试用例失败

用于判断预期结果与实际结果是否一致

3. 针对自动生成的代码进行修改



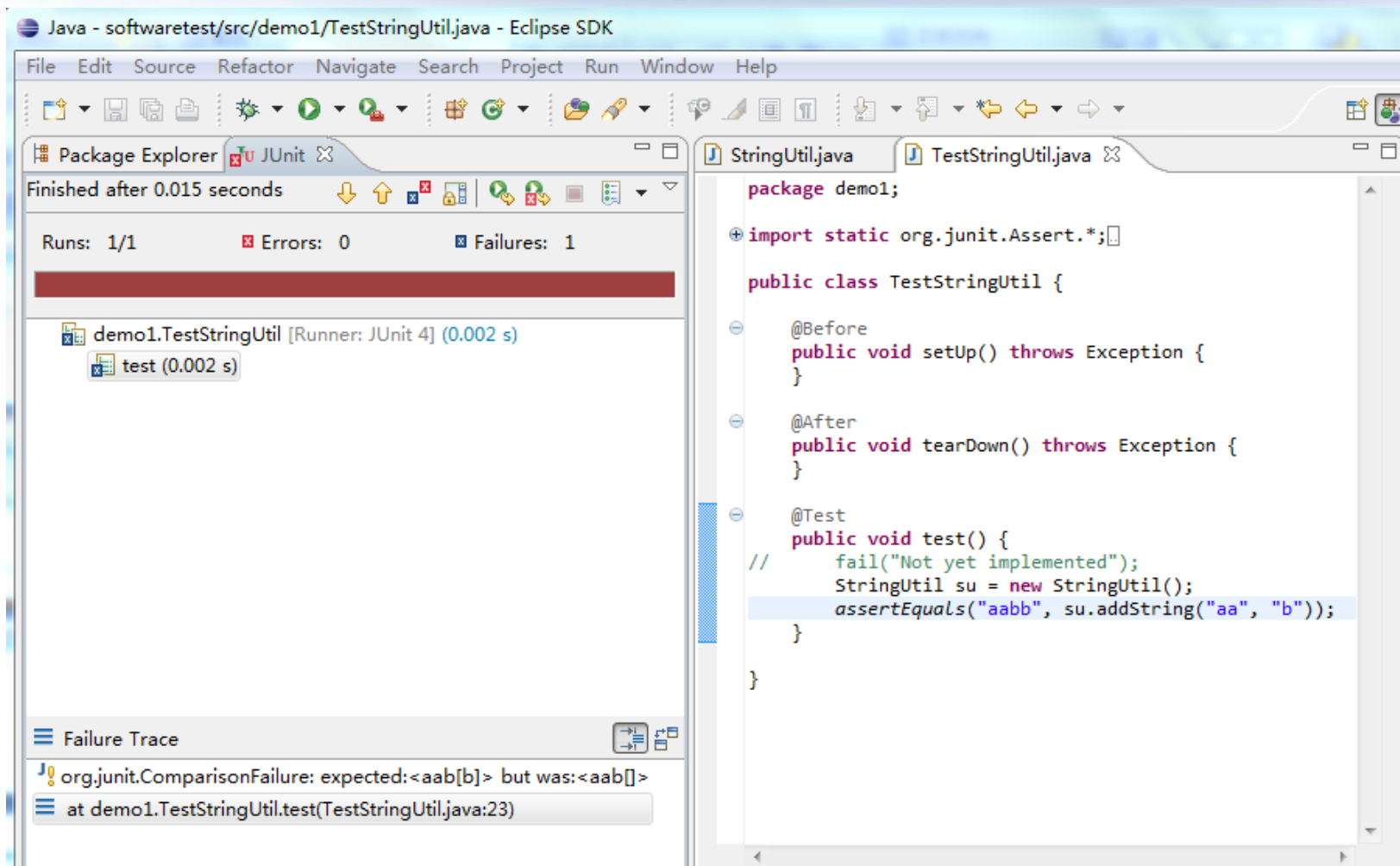
编写桩模块代码

桩模块代码

4. 测试用例执行通过



测试用例执行失败



测试失败会显示
红色的信息

JUnit的优势

- 使测试代码与产品代码分开
- 针对某一个类的测试代码通过较少的改动便可以应用于另一个类的测试
- 开源软件，可以进行二次开发
- 方便地对JUnit进行扩展
- 与开发环境Eclipse相互集成

单元测试的结果是什么？

□程序单元测试报告

- ✓测试用例的设计
- ✓程序单元的运行
- ✓运行结果情况
- ✓是否与预期相符一致

□谁负责撰写该报告

- ✓程序员

程序单元测试报告

什么时候实施单元测试？

- 根据设计文档，在程序代码编写之前设计好测试用例
 - ✓程序单元测试计划
- 程序代码写完之后开展程序单元测试

设计测试用例、制定测试计划与实际开展测试可在软件开发的阶段来开展！

3.6 单元测试的基本原则

- 由**程序设计人员**来完成
- 应在**基本功能/参数**上验证程序的正确性
- 应该产生**可重复、一致**的结果
- 具有**独立性**，单元测试运行/通过/失败不依赖于别的测试
- 应该覆盖所有**代码路径**
- 应该集成到自动测试的框架中
- 必须和产品代码一起保存和维护

谁负责开展程序单元测试？

- 单元测试必须由最熟悉代码的人来写
- 程序单元代码的编写者完成程序单元测试
- 程序员必须保证自己编写程序的质量

谁写的代码谁自己来负责单元测试

思考和讨论

- 测试用例的设计是否可以在编码阶段来完成?
- 为什么单元测试由程序员来完成, 集成测试和确认测试等由软件测试工程师来完成?



讨论

3.7 黑盒测试

□思想

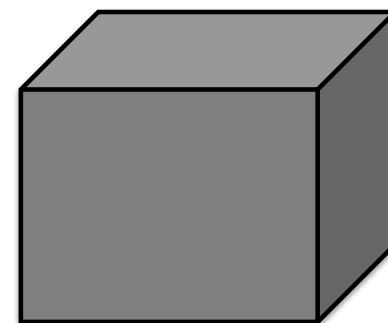
- ✓根据已知的**程序功能和性能**(而非内部细节), 设计测试用例并通过测试检验程序的每个功能和性能是否正常

□依据

- ✓程序的**功能和性能描述**

□特点

- ✓知道程序功能和性能, 不必了解程序内部结构和处理细节



只清楚模块的外在功能, 不清楚其内部的控制逻辑和算法

黑盒测试发现的缺陷类型

□测试软件系统是否**满足功能要求**

- ✓不正确或遗漏的功能
- ✓模块接口的错误
- ✓数据结构或外部数据库访问错误
- ✓性能的错误
- ✓初始化和终止条件错误

黑盒测试的特点

- **黑盒测试与软件如何实现无关**，如果软件实现发生变化，测试用例仍然可以使用
- **黑盒测试用例的开发可以与软件实现并行进行**，能够缩短软件开发周期

黑盒测试-等价分类法

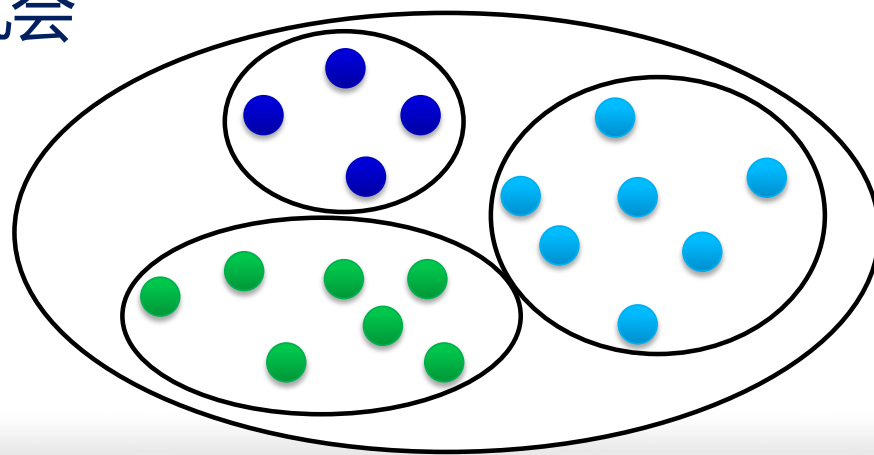
□思想

- ✓把程序的**输入数据集合**按输入条件划分为若干个**等价类**
- ✓每一个等价类对于输入条件而言为一组有效或无效的输入
- ✓为每一个**等价类**设计一个**测试用例**

□优点

- ✓减少测试次数，不丢失发现错误的机会

**每个等价类中的
数据具有相同
的测试特征**



等价分类法的基本原则

□ 输入条件为一范围

- ✓ 划分出三个等价类
- ✓ (1) 有效等价类(在范围内), (2) 大于输入最大值, (3) 小于输入最少值

□ 输入条件为一值

- ✓ 划分为三个等价类
- ✓ (1) 有效, (2) 大于, (3) 小于

□ 输入条件为集合

- ✓ 划分二个等价类
- ✓ (1) 有效(在集合内), (2) 无效(在集合外)

□ 输入条件为一个布尔量

- ✓ 划分二个等价类
- ✓ (1) 有效(此布尔量), (2) 无效(布尔量的非)

等价分类法示例

□ $z = \text{func}(x, y)$:

✓ 当 $0 < x < 1024$ 且 $y = 0$, $z = -1$

✓ 否则, $z = x * \lg(y)$

□ 关于 x 的等价类

✓ $(0, 1024)$

✓ $(-\#, 0]$

✓ $[1024, +\#)$

□ 关于 y 的等价类

✓ $\{0\}$

✓ $(-\#, 0)$

✓ $(0, +\#)$

设计的9个测试用例

- ① $X=1, y=0, z=-1$
- ② $X=1, y=-1, z=**$
- ③ $X=1, y=1, z=**$
- ④ $X=0, y=1, z=**$
- ⑤ $X=0, y=-1, z=**$
- ⑥ $X=0, y=1, z=**$
- ⑦ $X=2000, y=0, z=**$
- ⑧ $X=2000, y=-100, z=**$
- ⑨ $X=2000, y=200, z=**$

黑盒测试-边界值分析法

□输入条件是一范围(a,b)

✓ a,b以及紧挨a,b左右的值应作为测试用例

□输入条件为一组数

✓ 选择这组数最大者和最小者，次大和次小者作为测试用例

□如果程序的内部数据结构是有界的

✓ 应设计测试用例使它能够检查该数据结构的边界

软件更容易在输入的边界上出现错误

边界值分析法示例

□ $z = \text{func}(x, y)$:

✓ 当 $0 < x < 1024$ 且 $y = 0$, $z = -1$

✓ 否则, $z = x * \lg(y)$

□ 关于 x 的等价类-6个

✓ -1, 0, 1

✓ 1023, 1024, 1025

□ 关于 y 的等价类-3个

✓ -1, 0, 1

测试用例18个

- ① $X=1, y=0, z=-1$
- ② $X=1, y=-1, z=**$
- ③ $X=1, y=1, z=**$
- ④ $X=0, y=0, z=**$
- ⑤ $X=0, y=-1, z=**$
- ⑥ $X=0, y=1, z=**$
- ⑦ $X=-1, y=0, z=**$
- ⑧ $X=-1, y=-100, z=**$
- ⑨ $X=-1, y=200, z=**$
- ⑩

3.8 基于CASE的软件测试

- 测试数据产生器
- 运行测试程序
- 产生测试报告

基于大模型的软件测试

- 自动生成测试用例
- 自动生成测试代码



示例：基于ChatGPT的测试用例生成

- 生成四个测试用例
- 说明测试的目的、步骤和预期结果
- 考虑对应场景下的边界条件和异常情况
- 提供测试代码示例

You
请为上述“票务服务”组件的车票查询功能生成测试用例。

ChatGPT
为了确保“票务服务”组件的车票查询功能正常工作，我们需要编写一些测试用例来覆盖可能的场景。以下是一些针对车票查询功能的测试用例示例，旨在验证系统能够正确处理各种输入，并提供预期的输出。

准备工作

在编写测试用例之前，请确保你的开发环境中已经包含了Spring Boot Test依赖，以及JUnit和Mockito等测试库。

测试用例

- 1. 正常情况下的车票查询**
 - 目的：验证当输入有效的出发站、到达站和出发日期时，能够正确返回匹配的车票信息列表。
 - 步骤：
 - 1.1. 预设一个已知的出发站、到达站和出发日期。
 - 1.2. 调用车票查询服务。
 - 1.3. 验证返回的车票信息列表符合预期。
 - 预期结果：服务返回与输入条件匹配的车票信息列表。
- 2. 查询未来日期的车票**
 - 目的：验证系统能够处理对未来日期车票的查询请求。
 - 步骤：
 - 2.1. 输入一个未来的出发日期。
 - 2.2. 调用车票查询服务。
 - 2.3. 验证返回结果正确。
 - 预期结果：服务返回未来日期的有效车票信息列表。
- 3. 查询不存在的路线**
 - 目的：验证当查询不存在的出发站到到达站的路线时，系统能够正确处理。
 - 步骤：
 - 3.1. 输入一个不存在的出发站到到达站的路线。
 - 3.2. 调用车票查询服务。
 - 3.3. 验证返回结果为空或符合预期的错误信息。
 - 预期结果：服务返回空列表或适当的错误信息。
- 4. 输入无效的日期**
 - 目的：验证当输入无效的日期时，系统能够正确返回错误或异常提示。
 - 步骤：
 - 4.1. 输入一个无效的出发日期（如：格式错误、过去的日期）。
 - 4.2. 调用车票查询服务。
 - 4.3. 验证系统返回错误或异常提示。
 - 预期结果：服务返回错误提示或抛出异常。

示例代码（使用JUnit和Mockito）

假设我们的服务方法签名如下：

示例代码（使用JUnit和Mockito）

假设我们的服务方法签名如下：

```
java Copy code  
  
List<TrainTicket> searchTickets(String departureStation, String arrivalStation, LocalDate departureDate)
```

以下是一个使用JUnit和Mockito的测试示例代码：

```
java Copy code  
  
import org.junit.jupiter.api.Test;  
import org.mockito.InjectMocks;  
import org.mockito.Mock;  
import org.springframework.boot.test.context.SpringBootTest;  
import java.time.LocalDate;  
import java.util.Collections;  
import java.util.List;  
import static org.mockito.Mockito.when;  
import static org.junit.jupiter.api.Assertions.assertEquals;  
  
@SpringBootTest  
public class TicketServiceTest {  
  
    @Mock  
    private TicketRepository ticketRepository;  
  
    @InjectMocks  
    private TicketServiceImpl ticketService;  
  
    @Test  
    public void testSearchTicketsReturnsValidList() {  
        // Arrange  
        String departureStation = "Station A";  
        String arrivalStation = "Station B";  
        LocalDate departureDate = LocalDate.now().plusDays(1); // Future date  
        TrainTicket ticket = new TrainTicket(); // Assume this is a valid ticket object  
        when(ticketRepository.findByDepartureStationAndArrivalStationAndDepartureDate(departureStation, arrivalStation, departureDate)).thenReturn(Collections.singletonList(ticket));  
  
        // Act  
        List<TrainTicket> result = ticketService.searchTickets(departureStation, arrivalStation, departureDate);  
  
        // Assert  
        assertEquals(1, result.size()); // Expect 1 ticket in the result list  
    }  
  
    // Additional tests...  
}
```

内容

1. 软件测试概述

✓ 软件测试的思想和原理

2. 软件测试的过程和策略

✓ 软件测试的活动及实施的方法

3. 软件测试技术

✓ 白盒和黑盒测试技术

4. 软件测试计划及输出

✓ 测试计划制定及测试结果



4.1 成立软件测试组织

□软件测试是一项**独立性的工作**

□成立**单独的软件测试小组**

- ✓成员由各个软件测试工程师组成
- ✓高效的开展软件测试，确保软件测试工作的权威性
- ✓软件测试小组也不受软件开发小组的管理，以确保软件测试的独立性和结果的客观性

□需在软件开发**早期**就成立软件测试小组

- ✓以便他们尽早地介入到软件测试工作之中，包括开展必要培训、了解软件项目的整体情况、掌握软件需求、制定软件测试计划等

4.2 制定和实施软件测试计划 (1/2)

- **标识符**: 用以标识本测试计划
- **简介**: 介绍测试的对象、目标、策略、过程和进程等方面的内容
- **测试项**: 描述接受测试的软件元素, 包括代码模块或质量属性
- **待测软件特征**: 说明接受测试的软件特征, 如软件需求等
- **免测软件特征**: 说明无需接受测试的软件特征, 如软件需求等
- **测试策略和手段**: 说明对软件进行测试的策略和手段
- **测试项成败标准**: 说明每个测试项通过软件测试的标准
- **测试暂停/停止的标准**: 说明软件测试暂停或停止的决策依据

制定和实施软件测试计划 (2/2)

- **测试交付物**：说明测试过程中或完成之后应该交付的软件测试制品
- **测试任务**：描述测试的主要任务
- **测试环境**：描述测试所需建立的环境
- **职责**：说明每项测试任务的主要负责人或团队及其职责
- **进度安排**：描述测试的过程及时间安排
- **成本预算**：估算软件测试的成本
- **风险与意外**：描述测试过程中可能存在的风险和可能发生的意外

4.3 软件测试的输出

- **软件测试计划**，描述软件测试的整体规划情况
- **软件测试报告**，记录软件测试情况及发现的缺陷
 - ✓ 软件单元测试报告
 - ✓ 软件集成测试报告
 - ✓ 软件确认测试报告
 - ✓ 系统测试报告等

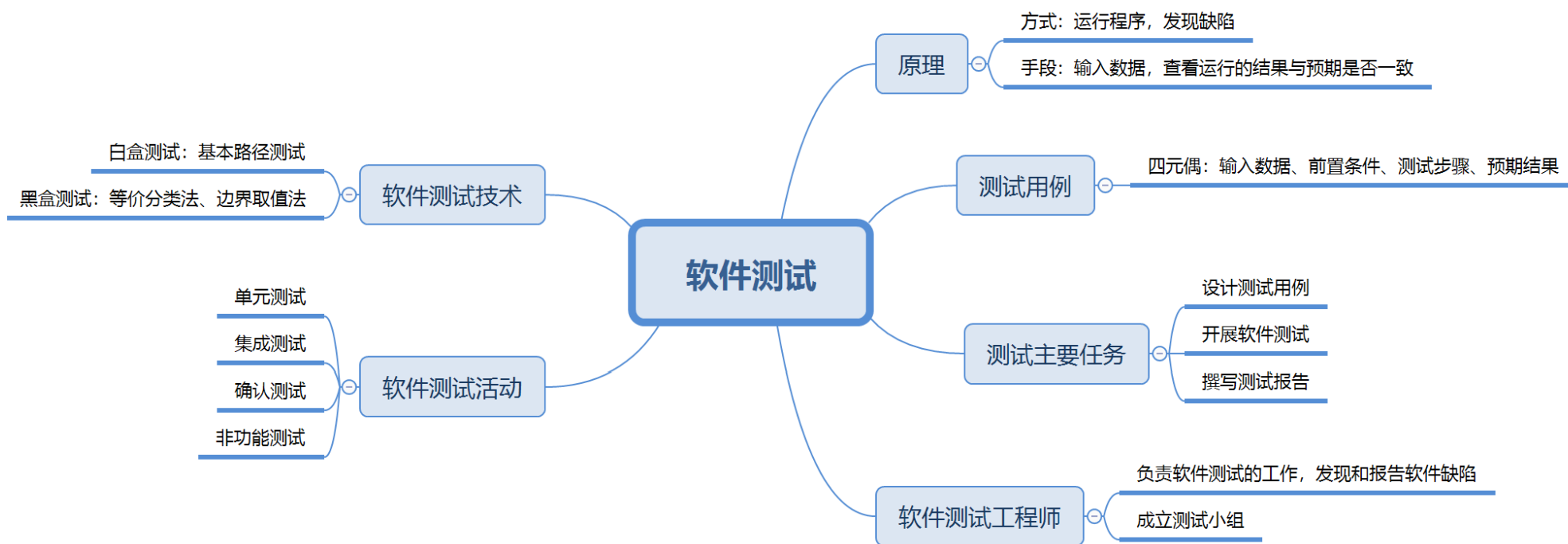
拓展阅读

□ 软件测试的艺术（第3版），Glenford J. Myers, Tom Badgett, Corey Sandler 著，张晓明，黄琳 译，机械工业出版社，2012.



这是软件测试领域的一本经典著作。该书结构清晰、讲解生动，简明扼要地介绍了诸多久经考验的软件测试方法，从软件测试的心理学和经济学入手，探讨了代码检查、走查与评审、测试用例的设计、模块单元测试、系统测试、系统调试等基本主题，以及极限测试、互联网应用测试等高级主题，全面展现了作者的软件测试思想。第三版在前两版的基础上，结合软件测试的新发展进行了内容更新，增加了可用性测试、移动应用测试以及敏捷开发测试等方面的内容

本章知识图谱



小结

□ 软件测试是为了**发现软件中的缺陷**

✓ 原理是设计和运行测试用例

□ 软件测试方法是**设计测试用例、运行测试代码、发现问题**

✓ 白盒测试、黑盒测试

□ 软件测试的**活动、过程和策略**

✓ 单元测试、集成测试、确认测试

□ 基于测试的结果来进行**纠错**

✓ 测试、调试和纠错

思考和讨论

如何改进和完善软件测试方法，
提高发现软件缺陷的能力？

